Persistent Case State for Agentic Legal Analysis: A Structured Approach to Complex Queries

Sigvard Bratlie

April 8, 2026

Contents

Abstract

Large language models (LLMs) are increasingly applied to legal case analysis, where a single matter may span hundreds of pages across contracts, correspondence, expert reports, and court filings. The standard approach — retrieval-augmented generation (RAG) — addresses context size constraints by retrieving semantically similar document chunks, but fails on queries that require synthesizing facts distributed across multiple documents. This thesis proposes and evaluates *structured extraction* as a complementary approach: as case documents are ingested, key entities — parties, events, claims, damages, and deadlines — are extracted into a persistent, structured *FactSheet* that supplements chunk-based retrieval for case-level queries.

A LangGraph-based conversational agent implementing this architecture is evaluated against a RAG-only baseline across two legal datasets, multiple session stages, and three model configurations. Results show statistically significant improvements in correctness ($p = 8.5 \times 10^{-3}$) and relevancy ($p = 3.4 \times 10^{-4}$) without a proportional increase in token consumption or inference latency. A consistent and statistically robust reduction in score variance is also observed across all five evaluation metrics, suggesting that structured state reduces the brittleness introduced by retrieval failures. The performance gap is most pronounced for smaller, less capable models, indicating a practical cost-quality substitution: a cheaper model paired with the structured architecture can approximate the performance of a larger model on RAG alone. These findings demonstrate that schema-driven extraction is an effective and cost-efficient strategy for improving factual accuracy and reliability in multi-document legal settings.

Chapter 1

Introduction

The legal industry is increasingly adopting large language models (LLMs) to assist lawyers in navigating the complexity of modern litigation. Legal cases are inherently multi-document affairs: a single matter may involve contracts, correspondence, expert reports, court filings, and evidence spanning hundreds of pages. As a case develops, new attachments arrive, earlier positions are revised, and the relevance of existing documents shifts. The central challenge for an LLM-based legal assistant is not merely reading these documents — it is maintaining a coherent, accurate understanding of the case as a whole, across all its documents and all its turns of development.

The standard approach to this problem is retrieval-augmented generation (RAG): rather than loading all documents into context, the system retrieves the chunks most semantically similar to the user's query. This is a practical solution to the context size problem, but it introduces a different and more fundamental limitation. Legal questions frequently require synthesizing facts that are distributed across multiple documents — a party's identity established in one contract, a disputed event described in correspondence, a damages figure appearing in an expert report. Chunk-based retrieval cannot produce this synthesis: it operates on isolated local fragments, and high semantic similarity to a query does not guarantee that the retrieved chunks contain the relational context needed to answer it correctly. The consequence is that naive RAG agents perform well on direct factual lookups but poorly on case-level reasoning.

An alternative — injecting all documents directly into a large context window — does not solve the problem either. Even as model context windows have grown to one million tokens and beyond, empirical evidence shows that effective utilization degrades well before the nominal limit, and model performance deteriorates for information buried deep in a long context. Beyond capacity, loading an entire case file for every query is impractical and costly. The goal should instead be to provide the minimum amount of information that maximizes relevance to the query at hand.

This thesis proposes and evaluates a third approach: *structured extraction*. As case documents are ingested, key facts — parties, claims, events, damages, governing law — are extracted and organized into a structured *FactSheet*: a persistent, machine-readable summary of the case. The FactSheet supplements chunk-based RAG for queries that require a case-level overview, providing relational context that retrieval alone cannot assemble. It is initialized at project creation and updated incrementally as new documents arrive, so the agent always has access to a coherent picture of the case without re-reading the full document history.

This thesis addresses the following research questions:

Main RQ: *Does schema-driven cross-document extraction into a structured case representation improve the factual accuracy of an agentic RAG system in multi-document legal settings, without incurring proportional increases in query-time cost?*

RQ1: *To what extent does structured extraction mitigate the cross-document retrieval limitations of naive RAG in legal case analysis?*
RQ2: *Does the structured case representation maintain or improve agent performance as document volume grows across case sessions?*
RQ3: *Can a structured extraction architecture compensate for reduced model capacity — enabling a smaller, less expensive model to achieve performance comparable to a larger baseline?*

The system is implemented as a LangGraph-based conversational agent. The main RQ is evaluated through a controlled comparison of a structured-state agent against a RAG-only baseline across factual quality and resource consumption metrics. RQ1 is addressed by examining performance on queries that require cross-document synthesis. RQ2 is examined through session-level trajectories as document volume grows. RQ3 is addressed by comparing performance gaps across models of varying capability.

The contributions of this thesis are: (1) the design and implementation of initialization and incremental update pipelines for structured legal case state management, (2) an evaluation framework comparing structured state agents against RAG-only and baseline approaches across multiple models and datasets, and (3) an empirical evaluation of how structured extraction affects performance across datasets, model configurations, and document volumes.

The remaining chapters are organized as follows: Chapter 2 presents the theoretical background on LLM context limitations, RAG and its limitations, and structured extraction approaches. Chapter 3 describes the methodology, system architecture, and evaluation design. Chapter 4 presents experimental results. Chapter 5 discusses findings and implications, and Chapter 6 concludes with a summary and directions for future work.

Chapter 2

Theory

This chapter builds the theoretical case for the approach taken in this thesis. The argument unfolds in three movements: first, why large language models alone are insufficient for multi-document legal case analysis; second, why retrieval-augmented generation — the standard remedy — carries fundamental limitations that make it inadequate for cross-document reasoning; and third, what the literature suggests as more capable alternatives, which together motivate the structured extraction approach evaluated in this thesis.

2.1 Large Language Models and Context Windows

Modern LLMs are built on the transformer architecture [1], which processes input as a sequence of tokens bounded by a fixed context window. Recent models advertise increasingly large context windows — some exceeding one million tokens — which may appear to solve the problem of large document corpora. This appearance is misleading for at least three reasons.

First, the nominal context size overstates effective capacity. Hsieh et al. introduce RULER, a benchmark measuring the *real* usable context of long-context LLMs, and demonstrate that performance degrades substantially before the advertised limit is reached [2]. Second, LLMs do not attend uniformly across the context: performance is highest when relevant information is at the beginning or end, and degrades for information buried in the middle — the *lost in the middle* phenomenon [3]. Third, and most practically, a legal case involving dozens of attachments spanning hundreds of pages cannot be loaded in full for every query without prohibitive cost and latency, regardless of model capability.

The implication is a design principle: rather than maximizing the amount of text in context, the goal should be to include the *minimum* amount of information while ensuring that information is as relevant as possible to the query at hand. This principle rules out naive full-document injection and motivates retrieval as the primary mechanism for selecting what enters context. The question then becomes: what are the limits of retrieval?

2.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) operationalizes the minimum-relevant-information principle by retrieving only the most semantically similar document chunks at query time, rather than loading entire documents [4]. Documents are chunked and embedded offline into a vector store; at inference time, the top-k most similar chunks are retrieved and inserted into the LLM context alongside the query.

For straightforward factual lookup — "what is the defendant's registered address?" — this works well. The answer lives in a single location in the document corpus, it is semantically similar to the query, and a chunk-based retriever will find it. The problems emerge when queries require understanding that spans multiple documents, or when the relevant information is not directly expressible as a similarity to the query string itself.

2.3 Limitations of Naive RAG for Complex Reasoning

2.3.1 Semantic Similarity Does Not Guarantee Relevance

The retrieval step selects chunks based on embedding similarity to the query. High similarity does not guarantee true relevance: documents may share keywords or surface features without addressing the underlying intent or logic of the query [5]. In legal contexts, where questions about causation, liability, or damages require understanding relational chains across documents, surface-level matching is an insufficient retrieval criterion.

Compounding this, injecting all retrieved chunks directly into context can produce fragmented or contradictory input that confuses the LLM and degrades answer quality [5]. The problem is not just what is retrieved but how it combines in context.

2.3.2 Cross-Document and Multi-Hop Reasoning

The more fundamental limitation is that the answer to a legal query often does not reside in one or two isolated chunks, but is distributed across multiple documents. Existing naive RAG systems are inadequate for queries that require retrieval and reasoning over multiple pieces of supporting evidence across documents [6]. In multi-document settings, relevant information is spread across sources and requires coherent synthesis: naively concatenating retrieved passages leads to fragmented or contradictory responses, a problem that is especially acute in legal and domain-specific QA where multi-hop reasoning is critical [7].

This limitation is structural, not a matter of tuning. A chunk embedding represents a local fragment of a single document. No similarity threshold or retrieval hyperparameter can make a chunk-based retriever produce a coherent synthesis of facts distributed across an entire case file.

2.3.3 Unstructured Retrieval and Missing Information

Standard RAG processes unstructured text passages without leveraging structured domain knowledge, and struggles with complex queries requiring multi-hop reasoning or domain-specific knowledge organization [8]. Unstructured retrieval also risks returning passages that contain non-atomic information — facts entangled with irrelevant surrounding text — that can mislead the LLM rather than inform it [8]. Chunk size amplifies this: small chunks provide insufficient context for understanding a passage, while large chunks dilute the semantic signal and retrieve off-topic content.

These three limitations — relevance mismatch, cross-document reasoning failure, and unstructured knowledge gaps — together define the core challenge this thesis addresses. A legal agent that relies solely on chunk-based RAG will, by design, struggle with case-level questions that require a coherent picture of the whole. This motivates approaches that move beyond isolated chunk retrieval toward structured, relational representations of case knowledge.

2.4 Graph-Based and Multi-Hop Retrieval

One line of work addresses RAG's limitations by enriching retrieval with relational structure. GraphRAG [9] builds a knowledge graph over the document corpus, enabling community-level summarization and structured traversal of entity relationships rather than relying on chunk similarity alone. HopRAG [10] extends this with explicit multi-hop reasoning, constructing logical chains across documents to answer queries that require synthesizing evidence from multiple sources.

These approaches directly target the cross-document reasoning failure described above. Rather than asking whether a chunk is similar to a query, they ask how entities and facts across the corpus are related. The factsheet approach implemented in this thesis can be understood as a lightweight, domain-specific variant of this idea: rather than constructing a general-purpose knowledge graph, it extracts and organizes precisely the information most relevant to legal case management — parties, claims, events, damages, and governing law — into a structured, queryable representation that supplements RAG for case-level queries.

2.5 Agentic Context Management

The final theoretical component concerns how structured knowledge is maintained over the lifecycle of a multi-turn conversation. Rolling summarization — periodically condensing conversation history into a compact summary — is a practical strategy for preventing context overflow in long sessions [11]. Stateful agent frameworks such as LangGraph enable persistent state management through checkpointing, allowing an agent to maintain structured representations that accumulate case knowledge across turns without requiring the full document history to remain in context [12].

In the system developed in this thesis, the factsheet is the primary persistent state artifact. It is initialized during document ingestion and updated incrementally as new attachments are processed and new facts confirmed, providing the agent with a structured, token-efficient overview of the case at every point in the conversation. The theoretical argument, then, is that this persistent structure addresses what RAG cannot: a coherent, relational picture of the case that is available without vector-based retrieval: the factsheet is loaded directly from the structured database, not assembled from semantically matched chunks at query time.

The following chapter describes how these theoretical motivations are translated into system design and how their combined effect is evaluated empirically.

2.6 Related Work

This section surveys prior work on the core problems this thesis addresses: overcoming RAG limitations in large multi-document corpora, structured information extraction for complex QA, and agentic LLM systems for professional knowledge work. The legal domain is the application context, but the architectural challenges — handling large, heterogeneous document collections with distributed factual content — are not unique to law. Related problems appear in financial due diligence, medical records analysis, and other high-stakes professional domains where information is voluminous, structured, and spread across many sources.

It is worth noting that most prior work on LLMs in legal settings concerns legal *reasoning* — statutory interpretation, case law analysis, and normative prediction [13, 14]. This thesis is not primarily concerned with legal reasoning ability. The focus is on the prior, more practical problem: how to make the factual content of a large case file reliably accessible to an LLM agent, regardless of which model underlies it. This distinction matters when interpreting the related work: findings about legal language models and legal reasoning benchmarks address a different problem, and this thesis fills a gap that those works do not cover.

2.6.1 Retrieval-Augmented Generation for Legal QA

RAG has emerged as the standard architecture for domain-specific document QA, and its application to legal settings has been explored in several directions. Louis et al. [15] demonstrate that RAG-augmented LLMs achieve competitive performance on long-form legal question answering while producing interpretable, citation-grounded responses. Their system retrieves relevant statutory passages and case law and incorporates them as context at inference time — the same basic mechanism used by the baseline agent in this thesis. Their work highlights both the strengths of RAG (traceable sourcing, factual grounding) and its limitations when queries require synthesising evidence from across a large corpus.

The fundamental tension identified in RAG-based legal systems is that legal questions rarely decompose into a single, retrievable passage. Multi-hop reasoning over distributed evidence is common: establishing liability may require linking a party's obligation (in a contract), its breach (in correspondence), and the resulting damage (in a financial report), none of which are co-located in the document corpus. MultiHopRAG [6] and HopRAG [10] address this limitation in general QA settings by explicitly chaining retrieval steps, constructing logical paths across sources. This thesis takes a complementary approach: rather than chaining retrievals reactively at query time, it extracts and pre-structures the key relational facts during document ingestion, making them available as a persistent case state without requiring multi-hop retrieval.

2.6.2 Structured Extraction and Knowledge Organisation

Extracting structured information from unstructured legal documents — parties, obligations, events, claims, and temporal relationships — has been studied as an information extraction problem long before the LLM era. Recent work has revisited this problem with LLMs: Jiang et al. [8] propose Retrieval and Structuring Augmented Generation (RSAG), which combines retrieval with LLM-driven structuring of retrieved content before generation. Their results show that structuring retrieved passages into typed entities before injecting them into the generation context consistently outperforms naive RAG on complex reasoning tasks. This provides independent empirical support for the core design hypothesis of this thesis: that structured, relational case state is more useful to an LLM than raw retrieved chunks.

GraphRAG [9] extends this idea to the full document corpus, building a knowledge graph over entities and their relationships to enable community-level summarization and structured traversal. The factsheet implemented in this thesis can be understood as a lightweight, domain-specific instance of the same principle: rather than constructing a general-purpose property graph, it organises only the entities most relevant to legal case management into a typed, relational structure. The practical trade-off is that a domain-specific schema can be designed to match exactly the queries a legal practitioner asks, at the cost of being less general than a full knowledge graph.

2.6.3 Research Gap

Taken together, the related work shows that (1) general RAG is insufficient for multi-document legal reasoning, (2) structured extraction improves performance on complex QA tasks, and (3) graph-based and multi-hop retrieval approaches partially address cross-document reasoning. However, the combination of persistent structured extraction with a conversational agent in a real legal case management setting — evaluated on authentic case documents across multi-session workflows — has not been studied in prior work. This thesis addresses that gap by implementing and empirically evaluating this architecture against a RAG baseline using real legal case files and expert-constructed ground truth.

Chapter 3

Method

3.1 System Architecture

The system is organized as a `uv` workspace containing one core module and three auxiliary modules. The `agent` package is the primary contribution of this thesis: it implements the full backend, including the LangGraph agent, document processing pipelines, vector store integration, and database management. The remaining packages are supporting tools: `ui` provides a Streamlit interface for non-technical users to interact with the agent; `evals` contains the evaluation harness used to benchmark agent configurations; and `data-viewer` is a lightweight helper for inspecting and editing evaluation datasets. The architecture description that follows concerns the `agent` module exclusively.

3.1.1 Overview

The backend is implemented as a **FastAPI** application (`agent/main.py`) that exposes a RESTful API with Server-Sent Events (SSE) for real-time response streaming. The frontend is a **Streamlit** application that communicates with the backend over HTTP, consuming streamed responses. Authentication is handled via **Supabase Auth** using JWT tokens, which are validated on each request through a `HTTPBearer` dependency in FastAPI.

3.1.2 Agent Layer

The core agent is implemented as a LangGraph `StateGraph` (see Figure 3.1). The graph has three nodes:

- `init` — Injects the system prompt as the first `SystemMessage` on a new conversation; is a no-op on resumption.
- `call_llm` — Assembles the full input payload (system prompt, factsheet context, conversation history, and any per-request attachment content), sanitizes it for provider compatibility, then streams the LLM response.
- `call_tool` — Executes all tool calls emitted by the LLM in the previous turn, with per-call caching to avoid redundant invocations within the same session.

Control flows as: `START` → `init` → `call_llm`. A conditional edge then routes to `call_tool` if the LLM response contains tool calls (looping back to `call_llm` afterwards), or to `END` if not. To manage long conversations within the model's context window, a rolling summarization strategy is applied every `sum_rate` messages (configurable, default 20) using a dedicated `Summarizer` module.

Conversation state is persisted using LangGraph's `AsyncPostgresSaver` checkpointer, backed by **Supabase PostgreSQL**, enabling session resumption and multi-turn dialogue across requests.

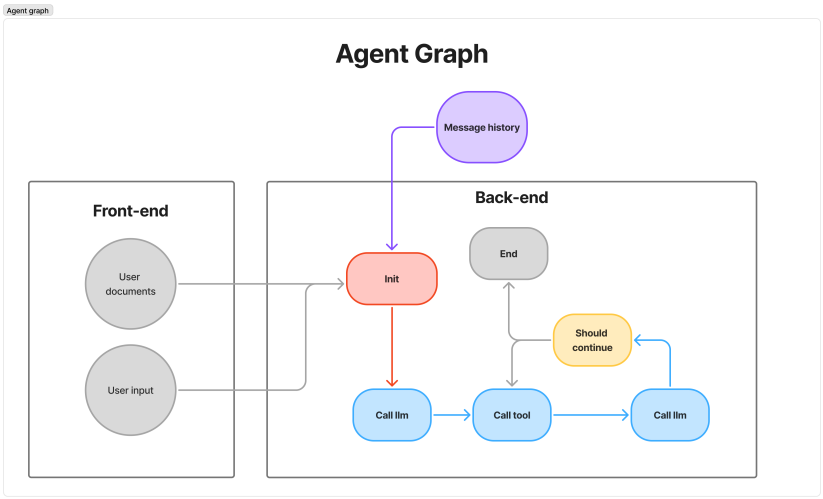

Figure 3.1: Agent graph: `init` → `call_llm` with a conditional loop through `call_tool`.

3.1.3 Tool Layer

The agent has access to a set of callable tools defined in `tools.py`. The full tool set (`TOOLS`) used in the primary agent configuration includes:

- `tavily_search` — Web search via the Tavily API.
- `read_attachments` — Fetches and returns the full parsed content of one or more documents by file ID. The parsed body is cached in the PostgreSQL database (populated during the document analysis pipeline); if not yet cached, the raw file is read from GCS and parsed on demand.
- `list_attachments` — Lists a project's attachments and emails (with metadata such as title, date, and category), optionally filtered by date range and significance. Returns file IDs that the model can pass to `read_attachments`.
- `show_elements` — Returns structured factsheet elements (events, parties, claims, damages, deadlines) for a project, optionally filtered by date range and significance level.
- `query_project_attachments` — Performs semantic retrieval (RAG) against project-specific document chunks indexed in the vector store.
- `query_laws` — Semantic retrieval against a corpus of Norwegian legislation, with optional filtering by law title.
- `read_specific_law` — Retrieves exact paragraph text from legislation by title and paragraph number via a parameterised BigQuery query.

An ablated tool configuration, `BASELINE_RAG_TOOLS`, is defined for evaluation purposes. It includes web search, law retrieval (`query_laws`, `read_specific_law`), and project document RAG (`query_project_attachments`), but excludes the structured factsheet tools and direct file-reading access.

3.1.4 Data and Storage Layer

Document persistence is handled through two complementary mechanisms. Raw files (PDF, DOCX, PPTX, EML) are stored in **Google Cloud Storage (GCS)**, where they can be retrieved on demand by the `read_attachments` tool when no parsed body cache is available. Parsed and chunked document content is embedded and stored in a persistent **vector store** for semantic retrieval. Structured project data — including the `FactSheet` and session metadata — is stored in a **PostgreSQL** database via the `SupabaseManager` CRUD interface.

3.1.5 FactSheet Schema

The *FactSheet* is the central persistent artifact of the system: a structured, machine-readable representation of the legal case that the agent maintains across the full lifecycle of a matter. It is stored relationally in PostgreSQL and loaded into context on each agent call, supplementing vector retrieval for case-level queries. The schema is defined using Pydantic `BaseModel` classes, which enforce field types and validation at runtime and serve as the structured output target for all LLM extraction calls.

The top-level `FactSheet` object aggregates five typed entity collections alongside a free-text background and case metadata. Table 3.1 summarises the complete schema, organised by entity type.

Table 3.1: FactSheet entity schema. All entity types share a `significance` field (*high / medium / low*) and a `file_id` / `email_id` back-reference to the source document.

Entity	Field	Description
FactSheet	`title`	Short case title (max 10 words)
	`background`	Narrative factual summary of the case
	`start_date`	Date the matter began
Party	`legal_name`	Full legal name of the party
	`role`	Functional role: *plaintiff, defendant, witness, expert, contractor,* etc.
	`entity_type`	*individual, company,* or *government*
	`role_description`	Free-text elaboration of the party's involvement
	`party_reps`	List of named legal representatives (name, role, contact)
Event	`event_name`	Short descriptive name
	`event_start_date`	Start date (partial dates coerced to first of month)
	`event_end_date`	End date if applicable
	`description`	Narrative description of what occurred
	`category`	e.g. *court_filing, contract_signing, communication*
	`disputed`	Boolean flag: is this event contested?
Claim	`category`	*principal_claim, counterclaim, objection, ancillary_claim,* etc.
	`legal_basis`	Statutory or contractual basis (e.g. *avtaleloven §36*)
	`factual_basis`	Key facts that support the claim
	`relief_sought`	What is being claimed (damages, injunction, etc.)
	`strength_assessment`	LLM assessment: *strong, moderate,* or *weak*
	`party_role`	Which party asserts this claim
Damage	`category`	*direct_losses, interest, consequential,* or *punitive*
	`amount`	Monetary amount if stated; `None` if unspecified
	`currency`	e.g. *NOK, USD*
	`basis`	Factual basis for the damage claim
	`supporting_evidence`	List of `file_id`s that evidence this damage
	`party_role`	Which party asserts this damage
Deadline	`deadline_date`	The deadline date
	`description`	What the deadline concerns
	`party_role`	Which party is subject to this deadline

Every entity carries a `significance` field (`"high"`, `"medium"`, `"low"`), assigned by the extracting LLM based on the entity's centrality to the case. This field is used at query time by the context compression layer (Section 3.2): only entities above a configurable significance threshold are included in the context injected into the LLM, keeping token consumption bounded as the factsheet grows.

Each entity also carries a `file_id` or `email_id` back-reference to the source document, enabling the agent to trace any extracted fact back to its evidentiary origin.

The schema is deliberately domain-specific: rather than a general-purpose knowledge graph, it captures precisely the entity types that legal practitioners reason over in a case management context — who the parties are, what happened and when, what is claimed and on what legal basis, what damages are asserted, and what procedural deadlines apply. This specificity is a design trade-off (discussed further in Section 3.2): the schema matches the query patterns of legal practitioners but requires adaptation for other legal domains.

3.1.6 Init and Update Pipeline

Project initialization (`POST /init-project`) and updates (`POST /update-project`) are handled by the `ProjectPipeline` class, which compiles a LangGraph `StateGraph` for each operation (see Figure 3.2). The graph exploits LangGraph's fan-out semantics to run independent branches in parallel, minimising total wall-clock time for the ingestion of large document sets.

Initialization pipeline. The following describes each node and its role in the `init` graph, in execution order:

- `collapse_emails` and `initialize_input` are launched concurrently from `START`. `collapse_emails` deduplicates EML threads by collapsing reply chains into canonical thread objects, preventing the same email content from being processed multiple times. `initialize_input` runs a lightweight LLM call over the user-provided case description to produce an initial structured seed for the factsheet: a short title, a factual background summary, and a list of named parties. This seed is used immediately as the factsheet starting state before any documents are processed.
- `extract_emails` follows `collapse_emails` and decodes each collapsed email thread, extracting inline attachments and populating the pipeline's document list with any files referenced in the email bodies.
- `parsing` and `storage` both fan out from `extract_emails` and run concurrently. `parsing` processes each document into plain text: PDFs are parsed with PyPDF2 (with OCR via `ocrmypdf` as fallback for scanned documents), DOCX files with `python-docx`, PPTX with `python-pptx`, and EML bodies directly from the collapsed thread objects. The parsed text is stored in the pipeline state for downstream use. `storage` uploads each raw file to Google Cloud Storage in parallel, populating the retrieval-by-ID cache used by the `read_attachments` tool.
- `embedding` follows `parsing` and chunks all parsed document text before indexing it into the persistent BigQuery vector store using Google's `gemini-embedding-001` model. This node runs independently and terminates at `END` once indexing is complete.
- `analyze` is a fan-in node that waits for both `parsing` and `initialize_input` to complete before executing. It is the core extraction node: all parsed documents and emails are dispatched to the LLM in size-bounded batches, and the model is asked to extract structured entities — events, claims, damages, deadlines, and document metadata — from each batch using the Pydantic schema described in Section 3.1.5. LLM calls are issued concurrently across batches via `asyncio.gather`, subject to the semaphore concurrency limits. The extracted entities from all batches are then merged into the pipeline state.
- `update_metadata` follows `analyze` and runs a further LLM call to synthesise a coherent case title, background narrative, start date, and primary party list from the full set of extracted data, overwriting the initial seed produced by `initialize_input`. This ensures the factsheet metadata reflects the complete document set rather than only the initial user description.
- `qc_analysis` fans out from `analyze` in parallel with `update_metadata`. It runs a quality-control pass over the extracted entity set, asking the LLM to identify logical inconsistencies, date conflicts, missing required fields, or implausible significance assignments. The results are logged and — if critical issues are found — surfaced to the frontend as warnings, but do not block the pipeline. This node provides a lightweight audit trail of extraction quality without introducing a blocking correction loop.

- `save` follows `update_metadata` and persists all extracted and updated entities to Supabase PostgreSQL via the `SupabaseManager` CRUD interface. Once saved, the factsheet is immediately available for agent queries.

After all terminal nodes (`embedding`, `storage`, `save`, `qc_analysis`) complete, a final status event is streamed to the frontend via SSE confirming that the project is ready for querying.

Update pipeline. The update pipeline shares the same graph topology as initialization, with one structural difference: a `load_project_data` node is inserted at `START` in parallel with `collapse_emails`. This node retrieves the existing factsheet from the database before the new documents are processed. The `analyze` fan-in then waits for both `load_project_data` *and* `parsing` to complete — substituting the `initialize_input` branch used during initialization — so that the LLM extraction step receives both the new document text and the full prior case state. This allows the model to resolve conflicts with previously extracted entities (e.g., duplicate events or parties already in the factsheet) rather than treating each update as an isolated extraction. The `save` node in the update pipeline merges new entities with the existing factsheet rather than overwriting it.

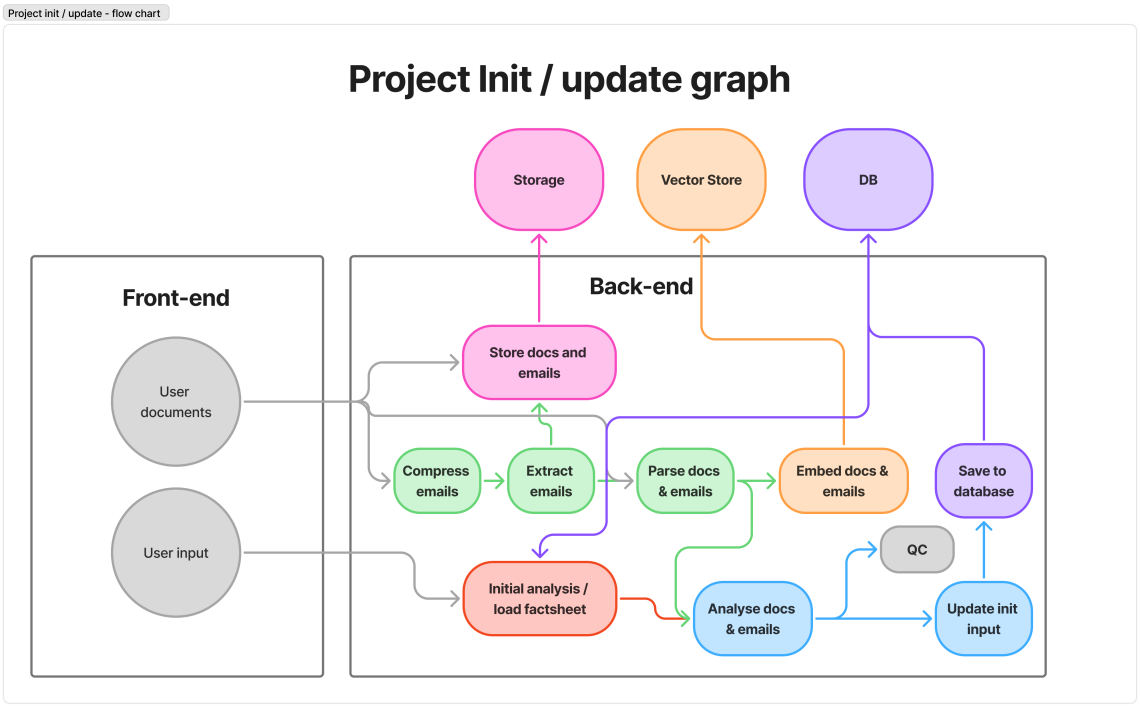

Figure 3.2: Init and update pipeline graph. `collapse_emails` and `initialize_input` (init) or `load_project_data` (update) run in parallel from `START`. After email extraction and document parsing, `analyze` acts as the central fan-in node. `update_metadata`, `qc_analysis`, `embedding`, and `storage` fan out from there, each terminating independently at `END`.

3.1.7 Clean Pipeline

The `ProjectClean` class provides a cleanup graph that can be triggered on demand (see Figure 3.3):

- **Clean Elements** (`compile_clean_elements`): `load_project` → `clean` → `save_results`. The `clean` node runs `_dedup_elements_node`, which dispatches all element types (events, parties, claims, damages, deadlines) to the LLM in configurable chunks concurrently via `asyncio.gather`. Crucially, the LLM is only asked to return the IDs of elements to keep — rather than rewriting full element objects — which minimises output token usage, reduces hallucination risk, and enables parallel deduplication of several hundred elements efficiently. Elements whose IDs are not returned are discarded, and results are saved back to the database.

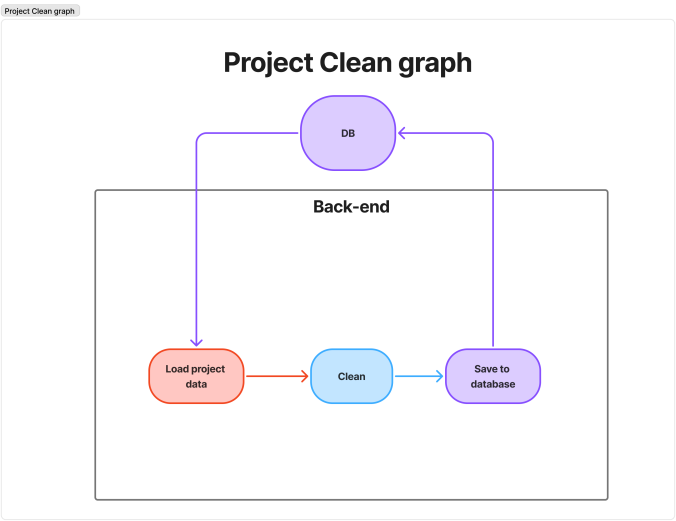

Figure 3.3: Clean pipeline graph: `load_project` → `clean` → `save_results`, used for both element deduplication and metadata rewriting.

3.1.8 LLM Configuration

The system supports multiple LLM providers through LangChain's `init_chat_model` interface. **Google Gemini** serves as the primary provider, with **OpenAI GPT** available as a secondary option. The system also supports a range of open-source models from providers such as Qwen (Alibaba) and Zhipu AI, accessed through compatible APIs. The active model is configurable per session request, allowing for flexible experimentation and evaluation across providers and model families.

3.1.9 Tech Stack Summary

Table 3.2 provides a concise overview of all major technologies used in the system, grouped by architectural layer.

Layer	**Technology**
Language	Python 3.13
Agent framework	LangGraph (StateGraph)
LLM orchestration	LangChain (`init_chat_model`)
LLM providers	Google Gemini (primary), OpenAI GPT (secondary)
Backend API	FastAPI + Uvicorn
Streaming	Server-Sent Events (SSE)
Frontend	Streamlit
Authentication	Supabase Auth (JWT)
Database	Supabase (PostgreSQL)
Vector store	BigQuery Vector Store (persistent, cross-session)
File storage	Google Cloud Storage (GCS)
Conversation checkpointer	LangGraph `AsyncPostgresSaver`
Document parsing	PyPDF2, ocrmypdf, python-docx, python-pptx
Embeddings	Google `gemini-embedding-001`
Web search	Tavily

Table 3.2: System tech stack.

3.2 Design Choices

The system architecture was guided by several pragmatic design choices, prioritizing development speed, flexibility, and cost-efficiency given the constraints of a master's thesis. By separating the descriptive architecture from the rationale, the core motivations for the selected technologies are outlined below.

3.2.1 Agent Layer and Orchestration

The core agent is implemented using **LangGraph** [12]. This framework was chosen primarily for its fast build time, which is essential given the limited time available for this thesis. It provides a clean `StateGraph` abstraction that is highly expandable for future extensions, such as adding dedicated reasoning nodes or document re-ranking stages. Furthermore, LangGraph is model-agnostic, providing built-in support for multiple LLM providers. This flexibility is crucial, allowing the system to easily switch between models or add new ones as desired for evaluation and experimentation.

3.2.2 Database and Storage Architecture

The system's persistence layer separates structured relational data, unstructured document storage, and semantic vector retrieval to optimize for specific access patterns and constraints.

Relational Storage (PostgreSQL via Supabase): PostgreSQL was selected for its flexibility and highly efficient reads across tables. The structured nature of the factsheet state requires strict enforcement of relationships through primary and foreign keys to maintain data integrity and orderly matching on IDs [16]. An initial attempt was made using a NoSQL database (Firestore); while easy and fast to set up, the lack of an enforced schema led to a loss of overview, with state inconsistencies and errors accumulating over time. Additionally, performing complex reads and combining data across different entities proved expensive and inefficient in NoSQL.

Supabase was chosen as the PostgreSQL provider because it is affordable, open-source, and seamlessly integrates other necessary products like Authentication. Having these features included out-of-the-box makes the development process straightforward and fast.

Database Schema

The database schema is designed to support the agent's state management and document metadata, while being flexible enough to accommodate future extensions. There are three main groupings of tables:

Project related tables
Projects
Parties
Events
Deadlines
Attachments
Emails
Claims (legal specific)
Damages (legal specific)

These tables are combined to form the factsheet state, which is the agent's internal representation of the case. The factsheet is updated and referenced throughout the agent's operation, and it is crucial that it is stored in a structured and consistent manner.

Session related tables
Sessions
Session Events
Session Attachments

Tables for storing document metadata and relationships.

Agent checkpoints
Checkpoints
Writes
Blobs

Tables for the agent's memory and conversation history.

File Storage (Google Cloud Storage): Raw documents (PDF, DOCX, EML, etc.) are stored in Google Cloud Storage (GCS). There were no specific architectural requirements for this other than cost-efficiency and reliability. GCS is affordable, integrates cleanly with other Google Cloud services already used in the stack, and provides durable object storage without requiring any dedicated infrastructure.

Relation to Graph-Based Retrieval: The combination of a relational schema and a vector store gives the system a hybrid retrieval architecture that shares structural similarities with graph-based RAG approaches such as GraphRAG [9]. The relational tables — parties, events, claims, damages, and deadlines — function as typed entity nodes, each linked back to the source document through foreign-key relationships and a `source_id` field. These links act as structured edges connecting extracted knowledge to its documentary origin, enabling the agent to reason not just about individual facts but about which documents substantiate them. Semantic search over the raw document chunks (via the vector store) serves the same role as embedding-based traversal in graph retrieval: it handles queries where the relevant content is not captured by the structured entities. The factsheet can therefore be understood as a domain-specific, relational graph over the case — not a general-purpose property graph, but a structured knowledge base with typed entities and cross-document connections, backed by semantic search for unstructured content.

Vector Store and Embeddings: Semantic retrieval relies on **Google's embedding model** (`gemini-embedding-001`). This model was specifically chosen for its strong multilingual capabilities; smaller and cheaper open-source models often struggle with RAG matching when queries or documents are not strictly in English, which is a critical requirement for handling Norwegian legal texts.

For the vector database, a two-tier approach is used. **ChromaDB** is utilized as an in-memory store for fast, session-scoped retrieval. For persistent, project-scoped retrieval, **BigQuery Vector Search** is employed. BigQuery was chosen because it is affordable and operates on an on-demand pricing model. The primary drawback of BigQuery is its high latency, meaning a significant amount of time is spent indexing and saving to the vector store. However, this trade-off is worthwhile: the alternative would require provisioning an always-running vector database server, which introduces high continuous costs and additional operational setup.

3.2.3 FastAPI and Server-Sent Events

The backend is exposed as a standalone REST API using **FastAPI** [17] to decouple it from the frontend. **Server-Sent Events (SSE)** are used for streaming responses because the agent pipeline is long-running and asynchronous. SSE provides a responsive user experience by pushing incremental status updates and partial results without the complexity of WebSockets, which are unnecessary for this unidirectional communication flow.

3.2.4 Concurrency, Reliability, and Tool Efficiency

Several engineering decisions were made to ensure the system operates efficiently under the latency and throughput constraints imposed by external APIs and large document volumes.

Async-first architecture. The entire backend stack is built on Python's `asyncio` event loop. FastAPI natively supports async request handlers, and all LangGraph pipeline nodes are declared as `async` coroutines. This allows the server to handle multiple concurrent sessions without blocking on I/O-bound operations such as LLM calls, database queries, or storage reads.

Semaphore-based concurrency control. To prevent overloading external services, four independent `asyncio.Semaphore` instances are used — one per resource class: LLM, database, storage, and vector store. Each semaphore has a configurable maximum concurrency value drawn from `config.toml` (e.g. `[async.llm] max_concurrent_requests = 20`). Pipeline nodes acquire the relevant semaphore before dispatching work, ensuring that bursts of parallel tasks are automatically throttled without busy-waiting or sleep loops.

Batch-wise and chunk-wise parallel processing. During project initialization, documents are grouped into batches before being dispatched for LLM analysis. Batching is driven by two configurable thresholds: a maximum byte-size per batch (`project.threshold`) and a maximum file count per batch (`project.max_attachments` / `project.max_emails`). Each batch is wrapped in an async task that acquires the LLM semaphore, and all tasks are dispatched concurrently via `asyncio.gather()`. The same chunk-wise strategy is applied during factsheet cleaning and deduplication: element lists (events, claims, damages, etc.) are split into fixed-size chunks (`project.clean.chunk_size`), each chunk is cleaned by a separate concurrent LLM call, and results are merged after all tasks complete. This approach keeps individual prompts within token limits while maximising throughput.

Rate limiting and 429 handling. To comply with provider-imposed rate limits and avoid HTTP 429 (Too Many Requests) errors, each LLM instance is wrapped with LangChain's `InMemoryRateLimiter`. The limiter implements a token-bucket algorithm: tokens are refilled at a configurable rate (`async.llm.requests_per_seco` and a burst capacity (`async.llm.max_burst_size`) allows short spikes. A single stateful limiter instance is shared across all coroutines that use the same model, so the rate limit is enforced globally rather than per-request.

Retry mechanism with exponential backoff. Transient API errors (e.g. network timeouts or momentary 5xx responses) are handled by a dedicated `apply_retry()` helper. It wraps any LangChain chain with `with_retry()`, configuring a stop condition (`retry_attempts` from config) and exponential backoff with random jitter (`wait_exponential_jitter=True`). The retry decorator is applied *after* structured-output binding so that validation failures are also covered. If all retry attempts are exhausted, the error is propagated and logged.

TOML-based configuration management. All tunable parameters — concurrency limits, rate limiter settings, retry counts, chunk sizes, model endpoints, and storage bucket names — are centralised in a single `config.toml` file. On startup, the file is parsed into a fully typed `AppConfig` object built with **Pydantic** and `pydantic-settings`. Nested configuration sections map directly to typed dataclasses (e.g. `AsyncConfig`, `ProjectConfig`, `VectorstoreConfig`), providing IDE auto-complete, validation at load time, and a single source of truth for the entire application. A `get_app_config()` singleton cached with `@lru_cache` ensures the config is parsed only once per process.

Tool interface design — string IDs over full objects. Agent tools are designed to accept minimal, flat inputs: all tools receive only string identifiers (e.g. `project_id: str`, `ids: list[str]`) rather than full domain objects. This keeps the JSON schema presented to the LLM small, reduces the token footprint of tool calls, and makes it straightforward for the model to construct valid invocations. Internal data fetching (loading full attachment records, querying the database) is performed inside the tool implementation, hidden from the model entirely.

Context compression via filtering and display shortening. A key design concern for the custom agent is managing the size of the structured context injected into each LLM call. As a case accumulates material, a naive approach of injecting the full factsheet verbatim would cause token consumption to grow unboundedly. Two complementary compression mechanisms are applied to mitigate this.

First, the `show_elements` and `list_attachments` tools accept optional `significance` and `date_range` parameters. These filters are implemented as PostgreSQL RPC functions on the Supabase backend and are applied server-side before any data is returned to the agent, ensuring that only elements above a configurable significance threshold and within the relevant time window are included in the context.

Second, the string representation of each structured element is deliberately abbreviated. Rather than serialising the full object with all fields, a compact display format omits less informative fields and truncates long free-text descriptions. Together, these two mechanisms reduce the effective model input by approximately 70–80 % compared to an unfiltered, full-fidelity serialisation of the same factsheet. This compression is a significant contributor to the finding that the custom agent's average token consumption remains statistically indistinguishable from that of the simpler baseline agent despite carrying substantially richer case context.

3.3 Evaluation Design

3.3.1 Datasets

The evaluation relies on two datasets developed specifically for this thesis in collaboration with legal experts from the law firm BAHR AS. Each dataset represents a real legal case processed through the Norwegian court system.

THRD-2021 is based on a publicly available district court case. The authentic court record — comprising the final submissions from both parties and the court's verdict — is used directly as part of the document set. The remaining case materials (correspondence, internal notes, technical reports, agreements, and supporting evidence) were not publicly available and were therefore synthetically reconstructed to match the factual matrix and legal reasoning documented in those authentic records. This hybrid approach provides a realistic document topology while respecting confidentiality.

TOSL-2024 is a fully authentic case for which explicit permission was obtained to use the original documents. It is substantially larger and more complex than THRD-2021: it involves more parties, a longer timeline, more contested legal issues, and a higher total document volume. The original case file includes close to a thousand emails; these were excluded from the evaluation dataset due to the prohibitive per-run cost and execution time they would introduce across repeated evaluation trials. Even after this exclusion, TOSL-2024 remains considerably larger and more complex than THRD-2021. Because TOSL-2024 is a real case with real documents, it more closely represents the conditions under which the agent would operate in production, and its results should therefore be given greater interpretive weight than those from THRD-2021.

The datasets are organized to simulate the chronological lifecycle of a legal proceeding. Each case is structured into several *sessions*, each containing a sequence of *queries*. This structure is designed to evaluate two critical capabilities of the agent:

1. **Contextual Awareness**: The ability to maintain a coherent understanding of the case as it evolves through different stages of litigation.

2. **Analytical Accuracy**: Each query is paired with a "gold standard" expected output, defined by legal experts. These references serve as the ground truth for measuring correctness, relevancy, and completeness.

By following a chronological order, the sessions require the agent to handle increasing amounts of information and maintain consistency over time, reflecting the professional requirements of a legal practitioner managing an ongoing case.

A significant effort was dedicated to the creation and refinement of these datasets, as no publicly available alternatives met the specific structural requirements of this evaluation. A survey of existing legal and large-scale QA datasets revealed that while many share some characteristics with the present work, none combine all the necessary elements. For instance, datasets like `Multi-LexSum` [18] offer large case files and expert summaries but lack the session-based, chronological query structure needed to evaluate temporal reasoning. Others, such as `CaseGen` [19], provide authentic, multi-section case samples but do not include the large, unstructured document collections typical of real-world litigation. Datasets like `EnronQA` [20] contain extensive email corpora but are not situated within a legal context. Finally, established benchmarks like `LegalBench`, `LexGLUE`, and others are primarily focused on legal reasoning over statutes or case law rather than fact management in an evolving case file. The evaluation thus necessitated a custom-built dataset format that included not only a large corpus of interconnected documents and emails but also a series of chronologically ordered, session-based queries designed to test contextual awareness over time.

The construction of the queries and their corresponding ground-truth answers was a meticulous process. Queries were designed to be fact-based, verifiable against the source documents, and challenging enough to require cross-document synthesis. Crafting the ground-truth answers presented a further challenge, particularly for queries where a range of correct responses is possible. The goal was to define a minimal, essential answer that captures the core facts, ensuring that the evaluation framework would not penalise models for providing additional, correct information. This required an in-depth manual review of all case documents for each query. After cleaning and removing anomalies, a total of 742 query-answer pairs were collected for the evaluation. Given the resource-intensive nature of this process, the expansion of these datasets and the creation of new ones remain important avenues for future work.

3.3.2 Evaluation Limitations and Caveats

Several factors limit the generalisability of the evaluation and should be considered when interpreting the results.

Absence of a plain LLM baseline. A natural comparison point would be an ordinary LLM queried without any specialised architecture — no document processing, no vector store, no RAG, simply the raw case files passed as context. This approach was explicitly attempted. In practice, the aggregated size of a realistic legal case file — spanning contracts, correspondence, technical reports, and legal submissions — immediately exhausted the rate limits of every model tested, well before a meaningful response could be produced. The sheer volume of document content triggers the providers' token-per-minute limits even for a single query, making repeated evaluation across sessions entirely infeasible. Feeding entire case files into a plain LLM context is therefore not a viable strategy for any of the models evaluated, and the minimum meaningful comparison is a RAG-enabled architecture. This is also why the evaluation compares the custom agent against a Baseline + RAG configuration rather than an unaugmented LLM.

Dataset size and the scalability trade-off. The evaluation uses a constrained number of cases and queries, as running a large-scale benchmark across multiple model-agent pairs with repeated trials becomes prohibitively expensive and time-consuming. This is, however, in tension with a fundamental property of the system: the agent is designed to improve with larger and richer document sets, as more material increases the factsheet's coverage and the quality of RAG retrieval. Evaluating on small datasets therefore likely *underestimates* the performance advantage of the custom agent in production-scale deployments.

Heterogeneity between cases. The two evaluation datasets differ substantially in origin, complexity, and document volume. THRD-2021 uses a hybrid document set (authentic court record supplemented by synthetically reconstructed case materials), while TOSL-2024 consists entirely of real documents. TOSL-2024 is considerably more complex: it covers more parties, a longer contested timeline, and a larger total document volume. This asymmetry means that performance differences across datasets may partly reflect document complexity rather than agent capability alone. However, since the agent is specifically designed for complex, large-scale cases, TOSL-2024 is the more ecologically valid test of its capabilities, and the dataset-level breakdowns in the results should be read with this in mind.

Exclusion of pipeline costs from resource metrics. The resource metrics reported in this evaluation — token usage and inference latency — cover only query-time activity: the LLM calls made by the agent when responding to a user query. The one-time cost of running the initialization and update pipeline (document parsing, entity extraction, embedding, and storage) is deliberately excluded.

This exclusion is methodologically correct for two reasons. First, including pipeline costs in a per-query resource comparison would produce a misleading figure on the evaluation datasets. Each dataset contains a small number of evaluation queries; amortising the full extraction cost across, say, four queries makes the cost per query appear orders of magnitude higher than it would be in practice. A production deployment handles hundreds or thousands of queries per case, over which the fixed extraction cost becomes negligible. There is no principled way to choose an amortisation denominator within the scope of this evaluation, and any choice would be arbitrary.

Second, the pipeline does not run at query time. It is an offline preprocessing step that can be triggered asynchronously — overnight, during off-hours, or on document arrival via integrations with document management systems and email. The practitioner does not wait for it. The relevant cost comparison is therefore query-time cost, which is what the reported metrics reflect.

Limitations of expected-answer evaluation. Model responses are graded against expected answers produced in collaboration with legal experts at BAHR AS. This approach carries two inherent risks: the expected answers themselves may contain inaccuracies, and — more fundamentally — many legal queries are open-ended, such that a correct and complete model response may differ substantially from the reference without being wrong. Three mitigations were applied to reduce this risk:

1. **Fact-based scope**: The evaluation is restricted to factual, document-grounded questions (e.g. "Who are the parties?", "What damages are claimed?"), deliberately excluding normative or predictive legal questions where correct answers are inherently ambiguous.

2. **Validatable content**: Questions are formulated so that their answers can be directly verified against the case documents, reducing the space for legitimate variation.

3. **Minimal-requirement framing**: Expected answers are written as *minimum* acceptable responses rather than exhaustive ideal answers, so a model response that covers the required facts is scored as correct even if it includes additional relevant information.

3.3.3 Evaluation Environment

Evalutation Framework:
The evaluation will be conducted using **DeepEval** [21], a framework for evaluating LLM outputs against reference answers. DeepEval allows for automatic scoring based.

Agent Configurations:
Two agent configurations are evaluated:

- **Custom Agent**: The full agent configuration with access to all tools, including structured factsheet access (`show_elements`, `list_attachments`), direct file reading (`read_attachments`), and RAG capabilities.

- **Baseline + RAG Agent**: An ablated version with access to web search, law retrieval, and project document RAG, but without direct file access or structured factsheet tools.

A plain LLM baseline without document retrieval was considered but excluded from the evaluation for the reasons described in Section 3.3.2: the volume of case material immediately saturates provider rate limits, making repeated evaluation impossible.

Model Configuration:
Since the scope of the thesis is not to evaluate the models themselves, but rather the design and architecture of the agent, a selection of four models is used for comparison. These include both closed-source and open-source variants to ensure the agent's performance is consistent across different provider architectures.

For open-source models, several were tested. Initial trials with smaller models, such as `Qwen3-Next-80B-A3B-Instruct` [22], demonstrated that they lacked the capacity to handle the complexity of the reasoning and extraction tasks, failing to produce coherent or useful outputs. Consequently, the evaluation focused on two of the top-performing large-scale open-source models available at the time of writing: `Qwen 3.5` from Alibaba and `GLM-5` from Zhipu AI. Both have a similar scale in terms of total parameters (approx. 400B), but differ significantly in their Mixture-of-Experts (MoE) architecture. Qwen activates 17B parameters per forward pass, whereas GLM-5 activates 40B, giving it a theoretical advantage in capacity that is later reflected in the results. Inference for these models was run via the `together.ai` [23] API, which was chosen for its production-grade scalability, affordable pricing, and seamless compatibility with the existing LangChain and OpenAI SDKs.

For closed-source models, Google's Gemini family was selected. `Gemini 2.5 Flash` provides a cost-effective and reliable baseline, suitable for extensive testing without incurring prohibitive costs. `Gemini 2.5 Pro` serves as the high-capability "gold standard" against which other models are measured. While other providers were considered and are supported by the system's architecture, Google's offerings were particularly advantageous for this evaluation due to their generous rate limits (per second, minute, and day) and large context windows. This was a critical practical consideration, as it minimised the risk of hitting API limits during the time-consuming data collection process, which could otherwise halt evaluation runs despite the presence of retry logic.

Table 3.3 lists the four models included in the evaluation, together with their parameter counts and providers.

The evaluation will be conducted across all datasets using consistent model-agent pairings. To account for the inherent stochastic nature of LLM outputs, each evaluation set (model + agent configuration) will be executed 3 times to ensure statistical significance in the results.

Table 3.3: Overview of Large Language Models used in evaluation.

Model Name	Type	Total Params	Active Params	Provider
Gemini 2.5 Pro	Closed	Unknown	Unknown	Google
Gemini 2.5 Flash	Closed	Unknown	Unknown	Google
Qwen 3.5 [24]	Open (MoE)	397B	17B	Alibaba
GLM-5 [25]	Open (MoE)	435.2B	40B	Zhipu AI

3.3.4 Metrics

The evaluation focuses on two categories of metrics: quality metrics that assess the accuracy and completeness of the agent's responses, and resource metrics that assess the computational cost of producing those responses. Together they allow the main research question to be evaluated jointly — whether any quality gains are proportionally offset by increased resource consumption.

Correctness, Relevancy, and Completeness (DeepEval). These three semantic quality metrics are evaluated using the **DeepEval** framework [21], which uses a separate LLM as an automated judge to score agent outputs against gold-standard reference answers. The judge model used throughout this evaluation is **Gemini 2.5 Flash**: it is cost-efficient enough to run at the scale required (742 query-answer pairs $\times$ 3 runs $\times$ 3 metrics), well-documented, and widely used in the LLM evaluation literature, making its scoring behaviour relatively well understood. Each response is evaluated independently on a continuous scale from 0 to 1.

Correctness measures whether the factual claims in the agent's response are accurate relative to the reference answer. It is the most diagnostic metric in this evaluation because it directly captures the architectural advantage under test: a structured factsheet provides the agent with pre-extracted, relational facts that vector retrieval alone may fail to surface from a large document corpus.

Relevancy measures whether the response addresses the question asked, rather than drifting to adjacent or tangential content. It functions primarily as a sanity check: as long as an agent engages substantively with the question, the response tends to be on-topic regardless of the underlying architecture. Cases where relevancy fails are therefore informative precisely because they indicate a retrieval or engagement failure, not merely a factual gap.

Completeness measures whether the response covers all facts required by the reference answer. It complements correctness by penalising omissions rather than factual errors: a response can be entirely accurate in what it states while still omitting important facts that the reference includes. Together, the three metrics span the accuracy–coverage–focus dimensions of answer quality.

A response is classified as a *pass* if its aggregate score meets or exceeds a predefined quality threshold; otherwise it is a *fail*. The proportion of queries receiving a passing score is reported as the **Passrate** metric.

BERTScore and Sentence-BERT Similarity. BERTScore [26] quantifies token-level semantic overlap between the agent's response and the reference answer using contextualised BERT embeddings. It computes *precision* (the fraction of generated tokens that are semantically present in the reference), *recall* (the fraction of reference tokens covered by the generated response), and an *F1* score that balances the two. Unlike the LLM-judge metrics, BERTScore requires no separate inference call and is insensitive to judge-prompt phrasing or judge model choice, making it a useful cross-validation signal for ruling out judge artefacts.

Sentence-BERT similarity complements BERTScore by capturing global embedding-level proximity between the two texts as a whole, rather than aligning individual tokens. The distinction matters because a response can score well on BERTScore — meaning its surface vocabulary overlaps with the reference — while omitting structured factual content that correctness and completeness evaluation would penalise. Including both metrics provides a more complete picture of semantic alignment and guards against the possibility that quality differences visible in the LLM-judge metrics are an artefact of embedding granularity.

Self-consistency. Self-consistency measures how semantically similar the agent's responses are when the same query is posed in different sessions. Because each agent run involves some inherent stochas-

ticity (temperature sampling, retrieval variance, session-level context differences), the same query may receive different responses across runs even if the underlying case state is identical. A high self-consistency score indicates that the agent converges reliably on the same answer regardless of session context; a low score suggests sensitivity to retrieval noise or prompt variation. This metric captures a dimension of reliability that per-query quality scores cannot: not just whether the agent answers correctly, but whether it does so predictably.

Self-consistency is computed as the mean pairwise cosine similarity of sentence embeddings across all runs of the same query:

$$\text{Self-Consistency} = \frac{2}{n(n-1)} \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \cos(\mathbf{e}_i, \mathbf{e}_j) \tag{3.1}$$

where n is the number of runs, $\mathbf{e}_i$ and $\mathbf{e}_j$ are the sentence embeddings of the i-th and j-th responses, and cosine similarity is defined as:

$$\cos(\mathbf{e}_i, \mathbf{e}_j) = \frac{\mathbf{e}_i \cdot \mathbf{e}_j}{\|\mathbf{e}_i\| \|\mathbf{e}_j\|} \tag{3.2}$$

Resource Metrics. Four resource metrics are recorded for each agent run: **inference duration** (wall-clock seconds from query submission to final response), **input token count**, **output token count**, and **LLM call count**. Input tokens directly drive API cost and context-window pressure; inference duration determines user-facing latency; LLM call count indicates how much internal orchestration each query requires. These metrics are analysed alongside quality scores to determine whether the custom agent's architectural overhead — the structured factsheet context injected on each call — imposes a measurable additional cost relative to the simpler baseline.

Additionally, an estimated cost per run is computed using each provider's advertised per-token prices:

$$\text{cost} = \text{input\_tokens} \times p_{\text{in}} + \text{output\_tokens} \times p_{\text{out}}$$

where p_{in} and p_{out} are the input and output token prices respectively. Because absolute costs differ substantially between datasets (TOSL-2024 involves larger document volumes and longer sessions than THRD-2021), raw cost figures are normalised to within-dataset percentile ranks for cross-model comparison.

3.3.5 Research Question Operationalization

Each research question is operationalized through a specific subset of the evaluation evidence collected.

Main RQ — accuracy improvement without proportional cost increase. The main RQ is evaluated by jointly examining quality metrics (correctness, relevancy, completeness, passrate) and resource metrics (input tokens, inference duration) across all queries and sessions. A positive finding requires both a statistically significant quality advantage for the custom agent *and* the absence of a significant resource cost increase. The one-sided Mann-Whitney U tests described below provide the formal statistical basis.

RQ1 — mitigation of cross-document retrieval limitations. RQ1 is addressed by examining the quality metrics specifically on queries that require synthesis across multiple documents and sessions. The dataset-level breakdown (Section 4) isolates TOSL-2024, which contains a larger document volume and more contested cross-document relationships, as the primary test case for cross-document reasoning. A larger performance gap on TOSL-2024 compared to THRD-2021 is interpreted as evidence that structured extraction specifically addresses the multi-document limitation.

RQ2 — performance stability across growing document volume. RQ2 is addressed by inspecting session-level performance trajectories. The datasets are organized chronologically into multiple sessions, each adding new documents. A custom agent that maintains or improves quality across sessions while the baseline degrades or stagnates provides evidence that the structured representation scales gracefully with document volume.

RQ3 — model capacity compensation. RQ3 is addressed by examining performance per model. The evaluation includes models of varying capability, from smaller open-source models (Qwen3 MoE 3B active parameters) to larger closed-source models (Gemini 2.5 Flash, Claude Sonnet). If the custom agent enables a smaller model to reach performance levels comparable to a larger baseline model, this constitutes evidence that structured context can partially substitute for raw model capacity.

3.3.6 A Priori Performance Expectations

The following expectations, grounded in the architectural differences between the two configurations, motivate the experimental design and provide a reference frame for interpreting the results.

Quality metrics over session progression. The central hypothesis is that the performance trajectories of the two architectures diverge as a case progresses. In the earliest sessions, the document set is small and context volume is modest; under these conditions the simpler RAG agent is expected to perform adequately, and the difference between architectures should be minimal. As subsequent sessions introduce new documents, parties, claims, and a growing timeline, the RAG approach becomes increasingly strained: retrieved chunks do not necessarily contain sufficient context for questions whose answers are distributed across multiple documents. The custom agent, by contrast, maintains a continuously updated factsheet and can draw on both structured case state and the same vector store as the baseline. The expected result is a positive or stable trajectory for the custom agent and a flat or degrading trajectory for the baseline.

Token usage over session progression. Token usage is expected to grow for both architectures as the case accumulates material, but for structurally different reasons. For the baseline, growth reflects increasing retrieval volume as more chunks are returned from a larger corpus. For the custom agent, growth reflects increasing *richness* rather than raw volume: the factsheet grows denser, but context compression keeps the effective input size in check. Output tokens are likewise expected to grow over time for the custom agent as richer context enables more thorough answers — a positive signal rather than an efficiency cost.

Inference time. Response latency is expected to increase with case progression for both architectures, proportionally to token growth.

3.3.7 One-sided Hypothesis Tests

Inspection of the metric distributions reveals pronounced left-skew with heavy concentration near 1.0 (see Figure 3.4), violating the normality assumption required by parametric tests such as the t-test. All hypothesis tests are therefore conducted using the one-sided **Mann-Whitney U test**, a non-parametric rank-based test that assesses whether one distribution tends to produce stochastically larger values than another, without assuming normality.

For quality/performance metrics (correctness, relevancy, completeness, passrate, self-consistency, BERTScore):

$$H_0 : \eta_{\text{custom}} \leq \eta_{\text{baseline+RAG}} \qquad H_1 : \eta_{\text{custom}} > \eta_{\text{baseline+RAG}}$$

For cost/efficiency metrics (token usage, inference time):

$$H_0 : \eta_{\text{custom}} \geq \eta_{\text{baseline+RAG}} \qquad H_1 : \eta_{\text{custom}} < \eta_{\text{baseline+RAG}}$$

Notation:

- η_{custom} = population median for the custom agent
- $\eta_{\text{baseline+RAG}}$ = population median for the baseline with RAG agent

All tests are one-tailed / one-sided.

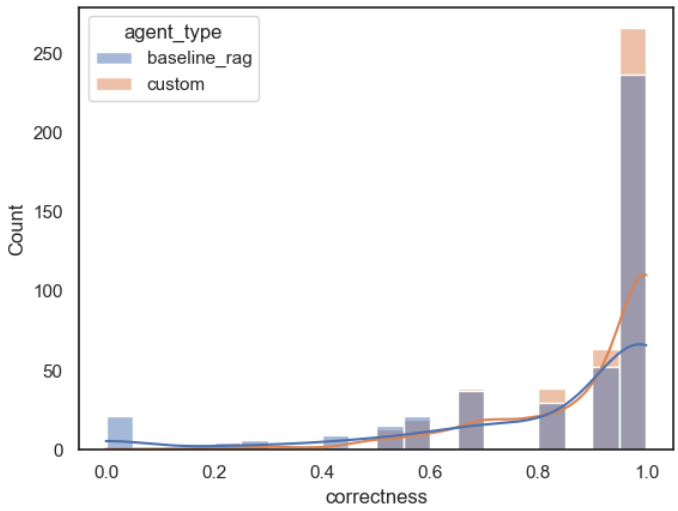

Figure 3.4: Distribution of correctness scores across all queries and sessions. The pronounced left-skew and heavy concentration near 1.0 violate the normality assumption, motivating the use of non-parametric Mann-Whitney U tests.

3.3.8 Levene's Test for Equality of Variance

The Mann-Whitney U test evaluates whether one distribution tends to produce stochastically *larger* values than another, but says nothing about whether the two distributions differ in *spread*. To test whether the custom and baseline agents exhibit significantly different score variability, Levene's test for equality of variance [27] is applied to each quality metric.

The test statistic is:

$$W = \frac{N-k}{k-1} \cdot \frac{\sum_{i=1}^{k} N_i \left(\bar{Z}_{i\cdot} - \bar{Z}_{\cdot\cdot}\right)^2}{\sum_{i=1}^{k} \sum_{j=1}^{N_i} \left(Z_{ij} - \bar{Z}_{i\cdot}\right)^2} \tag{3.3}$$

where:

- k = number of groups (here $k = 2$: custom and baseline)
- N = total number of observations across both groups
- N_i = number of observations in group i
- $Z_{ij} = |Y_{ij} - \bar{Y}_{i\cdot}|$, the absolute deviation of observation j in group i from its group mean
- $\bar{Z}_{i\cdot}$ = mean of Z_{ij} within group i
- $\bar{Z}_{\cdot\cdot}$ = overall mean of all Z_{ij}

Under the null hypothesis of equal variances, W follows an F-distribution with $k-1$ and $N-k$ degrees of freedom. A significant result ($p < 0.05$) rejects the null and indicates that the two groups differ significantly in spread. The hypotheses are:

$$H_0 : \sigma^2_{\text{custom}} = \sigma^2_{\text{baseline+RAG}} \qquad H_1 : \sigma^2_{\text{custom}} \neq \sigma^2_{\text{baseline+RAG}}$$

Levene's test is applied rather than Bartlett's test because it is robust to departures from normality [27] — a necessary property given the non-normal, left-skewed score distributions confirmed by Shapiro-Wilk ($p < 10^{-18}$ for every metric).

3.4 Development Tooling and AI Assistance

This section documents the tools and practices used during the development of this thesis — both the software infrastructure supporting the project and the role of AI assistance in the process.

3.4.1 Version Control and Repository Structure

The entire project is maintained in a single Git monorepo hosted on GitHub. This encompasses the agent source code, auxiliary modules (`ui`, `evals`, `data-viewer`), the evaluation datasets, and the LaTeX source for this thesis. Keeping all artefacts in one repository simplifies cross-referencing between the written thesis and the underlying implementation, and allows figures generated in the evaluation notebooks to be committed directly to the `thesis/figures/` directory — making figure updates a matter of running a notebook cell and committing the result. The only assets not stored in the repository are the datasets and case files, which are hosted in Google Cloud Storage due to their size and confidentiality.

Git's branching model and worktree support were used throughout development. Feature branches isolated experimental work from the stable main branch, and Git worktrees allowed parallel development — for example, running an evaluation trial on one worktree while implementing new features in another — without disrupting the working state.

3.4.2 Dependency Management

Python dependencies are managed with **uv**, a fast, standards-compliant package manager. `uv`'s workspace feature allows all four Python packages (`agent`, `ui`, `evals`, `data-viewer`) to share a single lock file and virtual environment root while maintaining separate dependency declarations per package. This was a key practical consideration for a monorepo of this scope.

3.4.3 Report Authoring

The thesis is written in LaTeX and compiled using `latexmk`, which automates the multi-pass build required for references, cross-references, and bibliography. The source is version-controlled alongside the codebase, enabling tight coupling between the report and the implementation: code snippets, figures, and table data are all traceable to specific commits.

3.4.4 Data Viewer

A lightweight **Streamlit** application (`data-viewer`) was developed to support dataset work and result inspection. It provides a browser-based interface for browsing and editing both training datasets and case files stored in GCS, displaying evaluation results, and inspecting analysis outputs. This reduced the overhead of dataset quality control and made it easier to identify issues during ground-truth annotation.

3.4.5 Use of AI Assistance

AI tools were used during the development of this thesis in accordance with NMBU's guidelines for the use of artificial intelligence in academic work [28]. All use is disclosed here as required.

The primary tool was **Claude Code** (Anthropic), a CLI-based AI assistant integrated into the IDE.

Code. Claude Code was used for code review, targeted refactoring, and implementation of cross-cutting infrastructure concerns — such as retry logic, rate-limit handling, and concurrency patterns — that are peripheral to the core research contribution. All architectural decisions, data model design, and system-level trade-offs were made independently by the author.

Report writing. AI assistance was used for reformulation and rewording of written content, improving clarity and maintaining a consistent academic register. It was also used to locate relevant background literature and identify applicable theoretical frameworks. All substantive content, arguments, and conclusions are the author's own, and all AI-assisted formulations have been critically reviewed and verified.

Dataset construction. For the THRD-2021 dataset, synthetic case materials (correspondence, internal notes, and supporting documents) were generated using an LLM, seeded from the authentic court record. This approach is discussed in Section 3.3.2 as a methodological caveat: while the synthetic documents were constructed to be consistent with the factual matrix of the authentic record, there is an inherent risk of logical inconsistencies between the real and synthetic materials, and this constitutes a recognised limitation of that dataset.

Ground-truth answers were reviewed and validated using AI-assisted quality control, in collaboration with legal experts at BAHR AS.

Chapter 4

Results

Data pre-processing. Prior to analysis, runs with a duration exceeding 600 seconds (10 minutes) were excluded as anomalies. These correspond to timed-out or otherwise pathological executions whose resource measurements would distort the distribution of duration and token statistics. All results reported below are computed on the filtered dataset.

4.1 Quality Metrics

4.1.1 DeepEval: Correctness, Relevancy, and Completeness

The joint distribution of correctness, relevancy, and completeness (defined in Section 3.3.4) across all queries and sessions is shown in Figure 4.1.

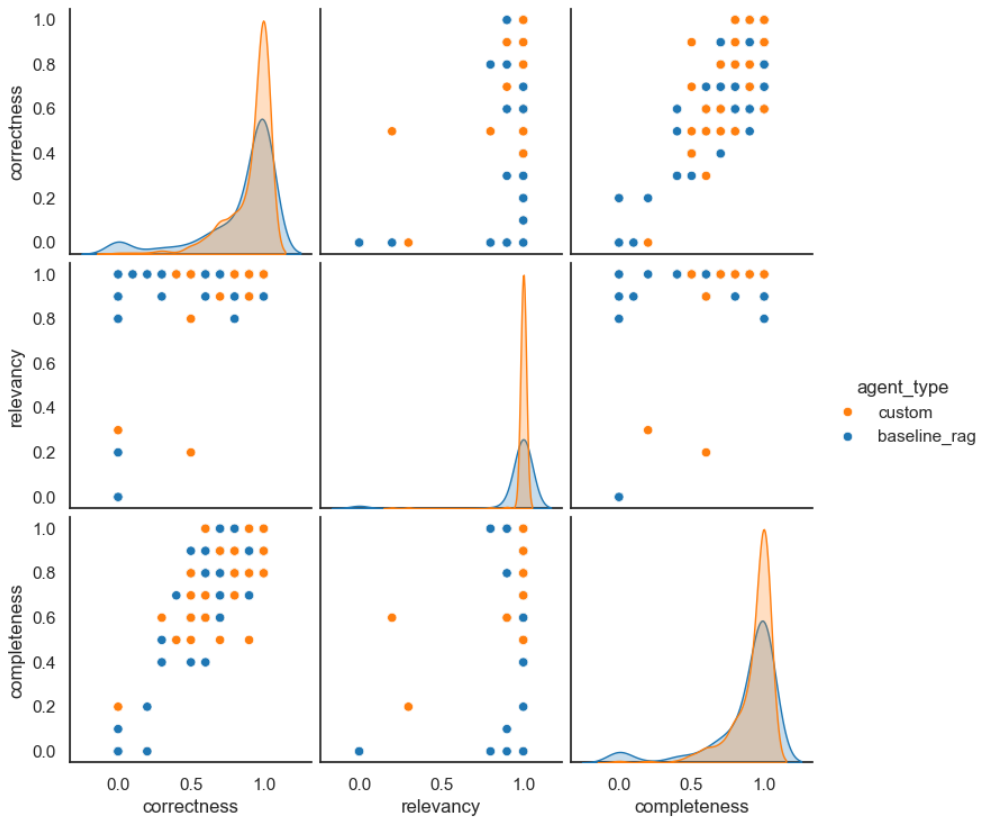

Figure 4.1: Pairplot of DeepEval quality metrics (correctness, relevancy, completeness). Diagonal panels show per-metric density estimates. The custom agent (orange) exhibits higher correctness and completeness, while relevancy is similarly high for both configurations.

Both distributions are left-skewed across all three metrics, with density concentrated near 1.0 — but the degree of concentration differs substantially. The custom agent's distributions are more tightly clustered near the upper end, while the baseline's are more spread, with a heavier low-scoring tail. This confirms that the custom agent's advantage is not driven by a small number of high-scoring queries but reflects a broad shift in the distribution. Relevancy scores are highly concentrated near 1.0 for both configurations — as long as an agent attempts to answer the question, the response tends to be on-topic regardless of architecture — though the baseline's low-scoring tail corresponds primarily to cases where retrieval fails entirely. Completeness follows the same pattern: both are left-skewed, but the custom agent's distribution is more concentrated near 1.0, while the baseline exhibits more low-completeness responses. Overall, the pairplot reveals greater consistency and reliability in the custom agent's responses across all three quality dimensions.

Session-level trajectories tell the same story longitudinally (Figure 4.2).

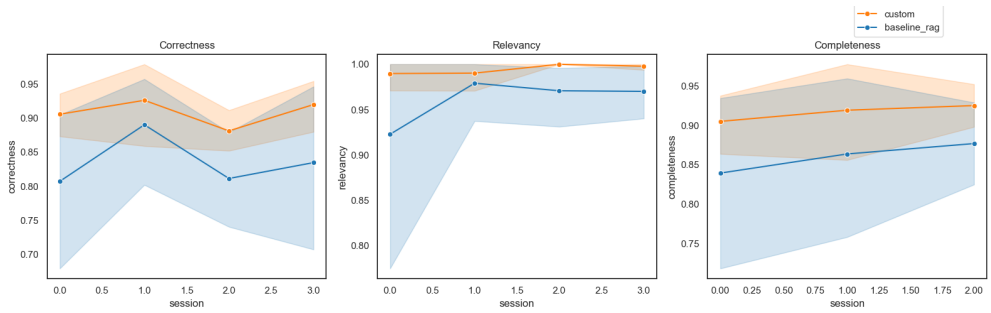

Figure 4.2: DeepEval quality metrics across sessions for each agent configuration.

Both agents broadly track each other through the session progression, but the custom agent remains consistently above the baseline throughout. Crucially, the variance of the custom agent's trajectory is substantially lower: where the baseline exhibits wider swings between sessions, the custom agent maintains a more stable performance profile. This session-level consistency reinforces the pairplot finding that the quality advantage is broad and reliable rather than driven by occasional strong queries.

Breaking the correctness trajectory down by dataset (Figure 4.3) reveals that the performance gap is not uniform across cases.

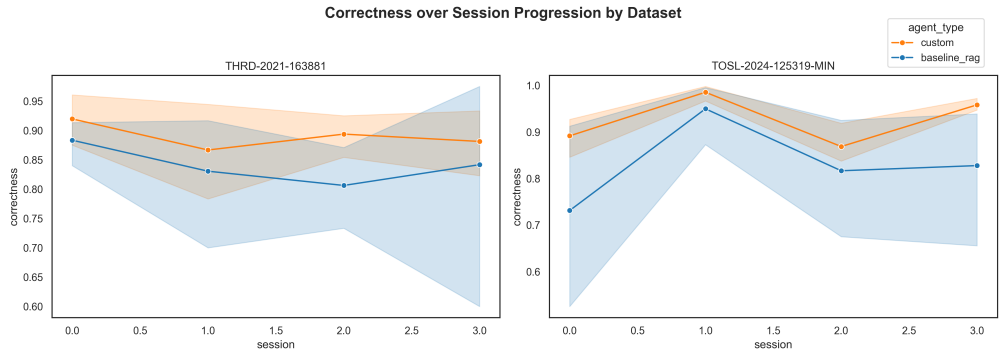

Figure 4.3: Correctness over session progression, split by dataset.

On TOSL-2024 — the fully authentic case with the larger document volume and greater factual complexity — the custom agent's advantage grows more noticeably as sessions accumulate. On THRD-2021, the custom agent also leads, but the gap is smaller and more stable. This pattern is consistent with the architectural hypothesis: the factsheet and structured retrieval tools confer the greatest benefit precisely when the case is large and complex, where naive RAG retrieval is most prone to missing relevant facts dispersed across many documents.

Figure 4.4 shows the same comparison as a violin plot, making the distributional differences and interquartile ranges directly visible.

Figure 4.4: Violin plot of DeepEval quality metrics (correctness, relevancy, completeness) by agent configuration. The custom agent exhibits narrower distributions with higher medians across all three metrics.

Figure 4.5 shows the distribution of binary pass/fail outcomes across all queries.

Figure 4.5: Distribution of pass/fail outcomes across all queries for each agent configuration, based on the DeepEval threshold. The custom agent achieves a higher pass rate.

The aggregate score distribution across all DeepEval metrics is shown in Figure 4.6.

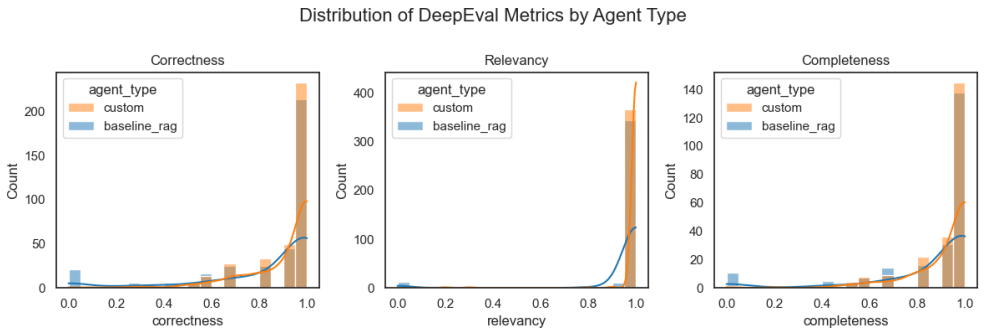

Figure 4.6: Aggregate distribution of all DeepEval metric scores. The custom agent's distribution is more concentrated near the upper end, while the baseline exhibits a heavier low-scoring tail.

The model-level breakdown (Figure 4.7) adds a further dimension.

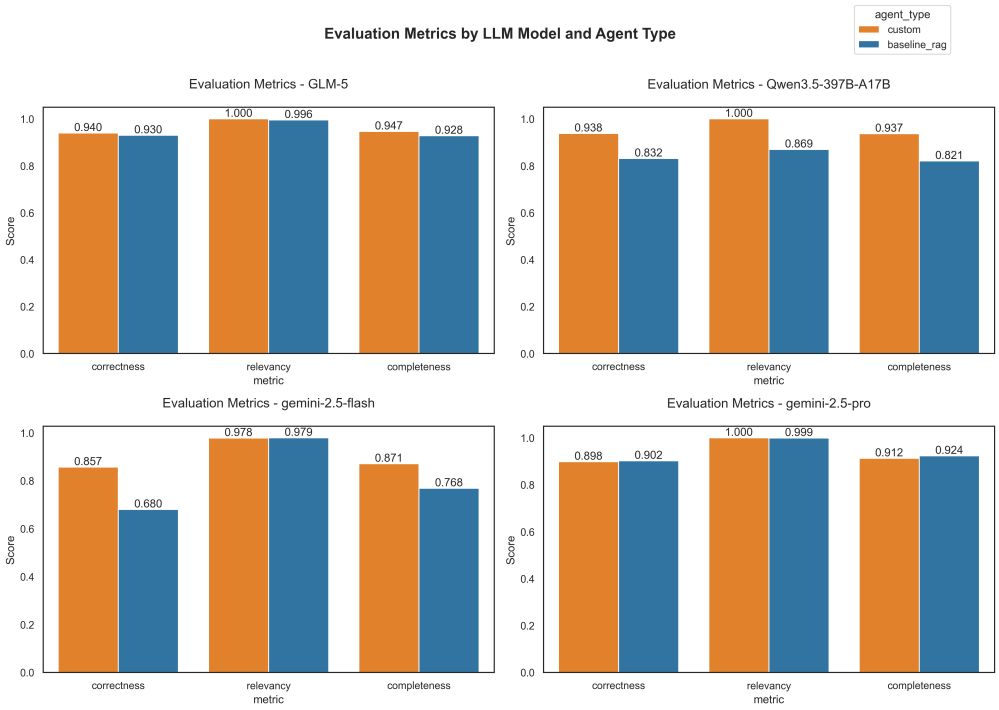

Figure 4.7: DeepEval quality metrics broken down by underlying LLM. The custom agent outperforms the baseline for every model tested, with the gap narrowing for the most capable model.

The custom agent outperforms the baseline across all three metrics for every model tested, with one exception: for the most capable model in the evaluation, the two configurations converge and the difference becomes negligible. For the smaller and less capable models, the margin is wider. This suggests that structured factsheet context acts as a capability amplifier whose marginal effect is greatest when the base model has limited capacity to compensate for noisy or incomplete retrieval. The implications of this model-dependent effect are discussed further in Chapter 5.

Dataset-level breakdown. Table 4.1 reports per-model scores for both agent configurations across each dataset. Table 4.2 condenses this into mean custom-minus-baseline differences, and Table 4.3 reports the variance of that gap across LLMs.

The dataset-level breakdown reveals a consistent asymmetry: the custom agent's advantage is substantially larger on TOSL-2024 than on THRD-2021 across all three quality dimensions. On THRD-2021, the mean correctness gain is +4.96 percentage points; on TOSL-2024 it more than doubles to

Table 4.1: Mean completeness, correctness, and relevancy by dataset, model, and agent configuration (baseline_rag and custom).

Dataset	Model	Comp. base	Comp. cust	Corr. base	Corr. cust	Rel. base	Rel. cust
THRD-2021	GLM-5	0.880	0.921	0.890	0.928	0.992	1.000
THRD-2021	Qwen3.5-397B	0.937	0.947	0.927	0.944	0.996	1.000
THRD-2021	gemini-2.5-flash	0.740	0.853	0.665	0.831	0.996	0.979
THRD-2021	gemini-2.5-pro	0.903	0.870	0.894	0.871	1.000	1.000
TOSL-2024	GLM-5	0.992	0.982	0.971	0.956	1.000	1.000
TOSL-2024	Qwen3.5-397B	0.700	0.926	0.727	0.927	0.733	1.000
TOSL-2024	gemini-2.5-flash	0.822	0.911	0.718	0.889	0.967	0.980
TOSL-2024	gemini-2.5-pro	0.956	0.956	0.911	0.924	0.998	1.000

Table 4.2: Mean difference (custom − baseline) per dataset, in percentage points. Positive values indicate the custom agent scores higher.

Dataset	Completeness	Correctness	Relevancy
THRD-2021-163881	+3.27%	+4.96%	−0.10%
TOSL-2024-125319-MIN	+7.60%	+9.24%	+7.06%

+9.24 pp. Completeness follows the same pattern (+3.27 vs. +7.60 pp), while relevancy is effectively neutral on THRD-2021 (−0.10 pp) but shows a clear positive gain on TOSL-2024 (+7.06 pp).

This asymmetry is consistent with the architectural hypothesis. TOSL-2024 is the fully authentic dataset with a larger and more heterogeneous document set; queries on this case require cross-document synthesis that naive RAG is most prone to fail. The factsheet's structured representation provides the greatest marginal benefit precisely where retrieval-only approaches are most likely to miss facts dispersed across many documents.

The variance table reinforces this interpretation. The variance of the per-model gap is uniformly higher on TOSL-2024 than on THRD-2021 across all metrics. This indicates that the advantage is less uniform across LLMs on the more complex dataset: some models benefit more from the structured context than others, and this interaction is amplified when the case is harder. The notable exception is gemini-2.5-pro on TOSL-2024, where the baseline already achieves high scores, leaving less headroom for improvement.

4.1.2 BERTScore

BERTScore and Sentence-BERT similarity (defined in Section 3.3.4) serve as a cross-validation signal that is independent of the LLM judge used for the DeepEval metrics. Because BERTScore measures surface-level semantic overlap, it is less sensitive to structured factual completeness than the DeepEval metrics: a response can score well on BERTScore while omitting specific facts that correctness and completeness evaluation would penalise. Agreement between the two metric families therefore provides stronger evidence that observed quality differences reflect genuine architectural effects rather than judge artefacts.

Table 4.3: Variance of the per-model custom−baseline difference per dataset, expressed as a percentage (×100). Higher variance indicates less uniform gains across LLMs.

Dataset	Completeness	Correctness	Relevancy
THRD-2021-163881	0.38%	0.67%	0.01%
TOSL-2024-125319-MIN	1.20%	1.18%	1.71%

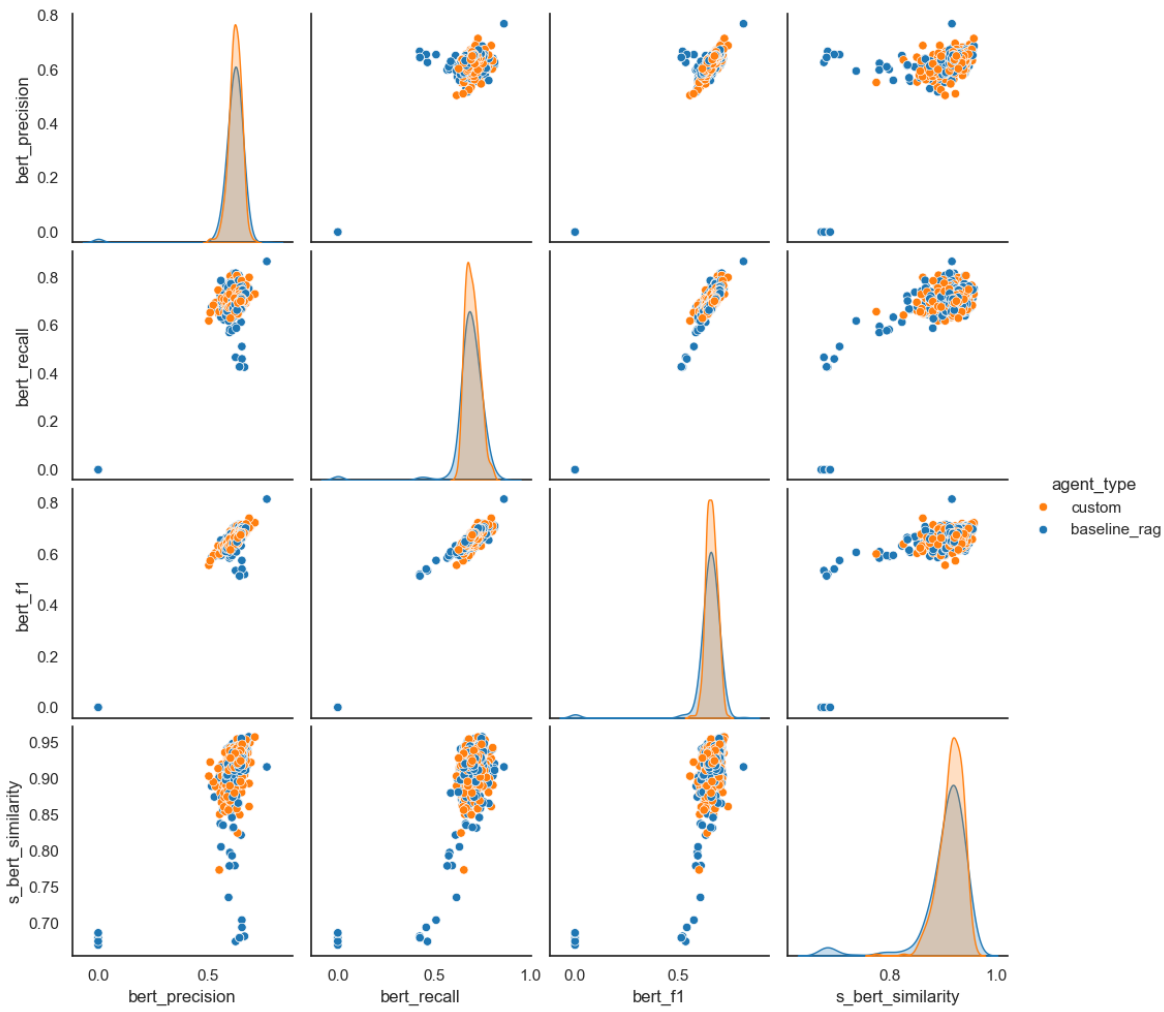

Figure 4.8: Pairplot of BERTScore sub-metrics (precision, recall, F1, and sentence-BERT similarity). Distributions for the two agent configurations largely overlap, indicating comparable surface-level semantic quality.

The distributions have considerable overlap between the two agent types, but the custom agent shows a higher peak with less variation and more consistent surface-level semantic similarity to the reference answers. The baseline has a wider spread with more low-scoring outliers, suggesting it occasionally produces responses less semantically aligned with the reference. The near-identical BERTScore distributions complement rather than contradict the DeepEval findings: both agents produce semantically coherent output, but the custom agent more reliably includes the specific factual content the gold-standard reference requires.

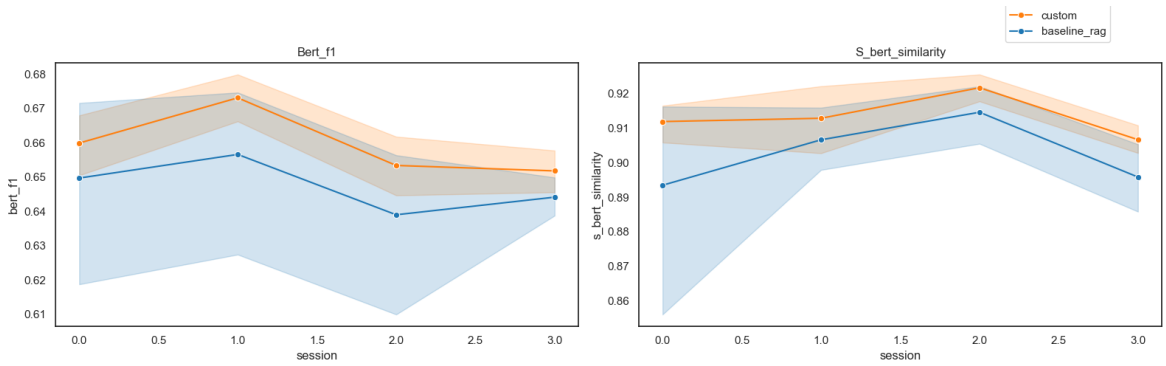

Figure 4.9: BERTScore F1 and sentence-BERT similarity across sessions. Trajectories are flat and closely aligned between configurations, with the custom agent showing marginally narrower confidence intervals.

Across sessions, BERTScore F1 and sentence-BERT similarity are relatively flat and closely aligned for both agent types, with only minor fluctuations. This is expected: surface similarity to reference answers does not change systematically as the case grows, since the phrasing of correct responses does not become more or less reference-like with document volume. The session-level divergence observed in the DeepEval metrics is therefore not an artefact of reference phrasing but reflects genuine differences in factual coverage.

4.1.3 Resource Metrics

Resource metrics (defined in Section 3.3.4) are analysed alongside quality scores to determine whether the custom agent's architectural overhead is justified by its performance gains.

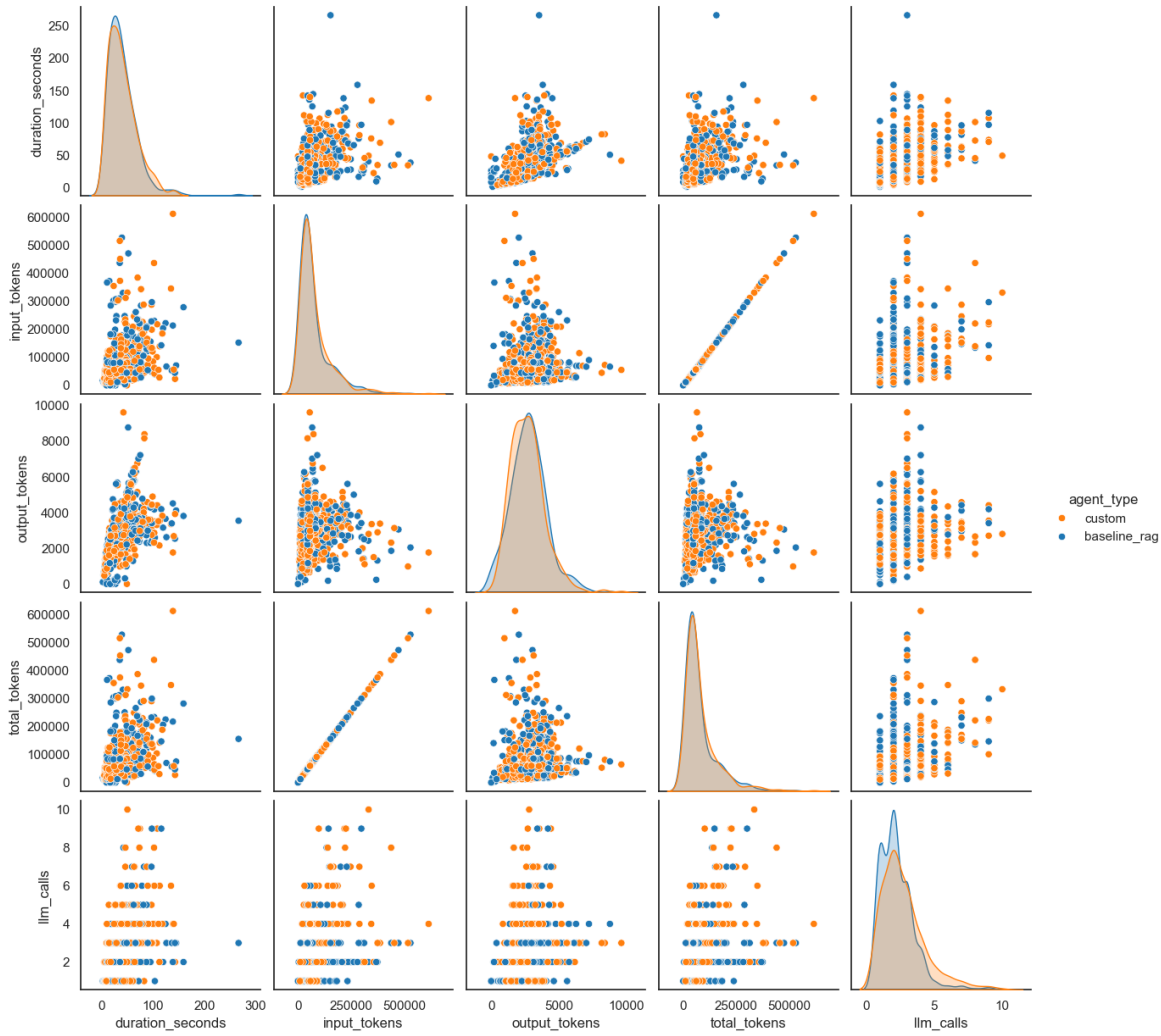

Figure 4.10: Pairplot of resource metrics (duration in seconds, input tokens, output tokens, total tokens, and LLM call count). The custom agent exhibits higher input token usage reflecting factsheet growth, while inference duration is more tightly bounded than the baseline.

The most notable pattern in Figure 4.10 is in input token usage. The custom agent spans a wider range, with some queries consuming upwards of 200,000 input tokens — consistent with the factsheet growing denser as the case accumulates material and vector retrieval returning richer chunks. The baseline clusters at lower input token counts, reflecting its simpler retrieval strategy, though it too exhibits high-token outliers for sessions with large document sets. Output tokens are similarly distributed, with the custom agent showing a slightly wider tail toward higher values, in line with the expectation that richer context enables more detailed answers. Inference duration is generally lower and more tightly concentrated for the custom agent; the baseline exhibits heavier tails at high latency, likely due to its unbounded document ingestion strategy. LLM call counts are comparable between configurations, typically in the range of 1–3 calls per query.

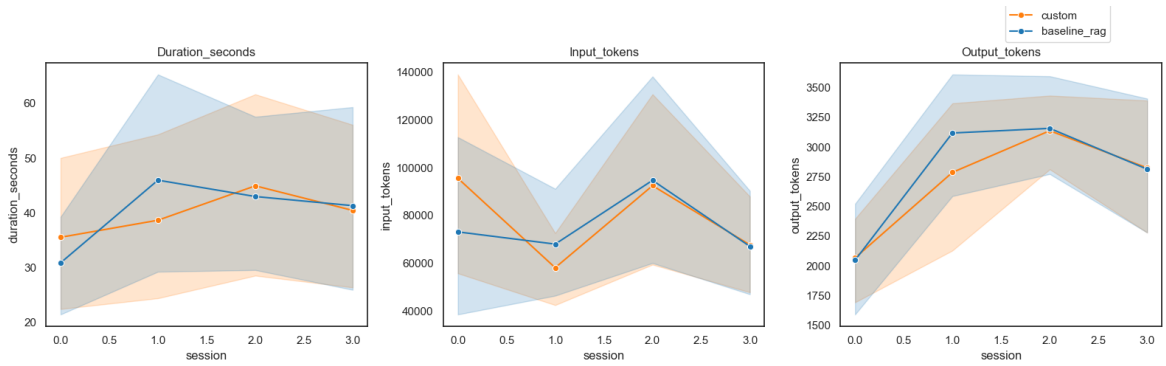

Figure 4.11: Resource metrics across sessions. Token counts and latencies fluctuate without a systematic trend for either configuration, and the two track each other closely throughout.

Figure 4.12 plots inference duration against correctness score for individual queries, illustrating that higher latency does not correspond to higher quality.

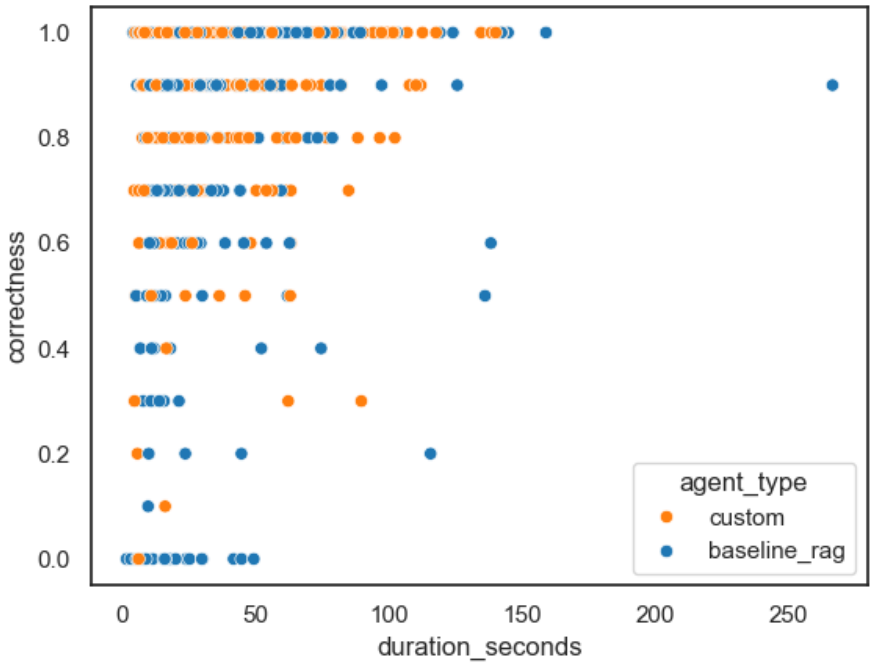

Figure 4.12: Scatter plot of inference duration (seconds) against correctness score per query. No positive correlation is observed, indicating that longer inference time does not reliably produce higher-quality responses.

Unlike the quality metrics, no consistent trend is visible in the resource trajectories over session progression. Token counts and latencies fluctuate without a systematic increase, and the two configurations track each other closely throughout (Figure 4.11). The absence of a growth trend is partly explained by the context compression mechanisms applied to the custom agent: as the factsheet grows, significance filtering and display shortening keep the payload from expanding proportionally.

Cost analysis. Absolute token counts are not directly comparable across datasets with different document volumes and session lengths. The total cost for TOSL-2024 is substantially higher than for THRD-2021, making direct comparison of absolute costs misleading. To enable model-level cost

comparisons, each run's estimated cost (input tokens × price-per-token + output tokens × price-per-token) is normalised to a percentile rank within its dataset:

$$\texttt{rank\_cost} = \text{rank}_{\text{pct}}(\text{total\_cost}) \quad \text{grouped by dataset}$$

A rank of 1.0 means the most expensive configuration within that dataset; 0.0 means the cheapest. This makes cost comparable across datasets of different scales without requiring absolute price estimates for every provider.

Figures 4.13–4.15 plot cost rank against each quality metric for individual queries.

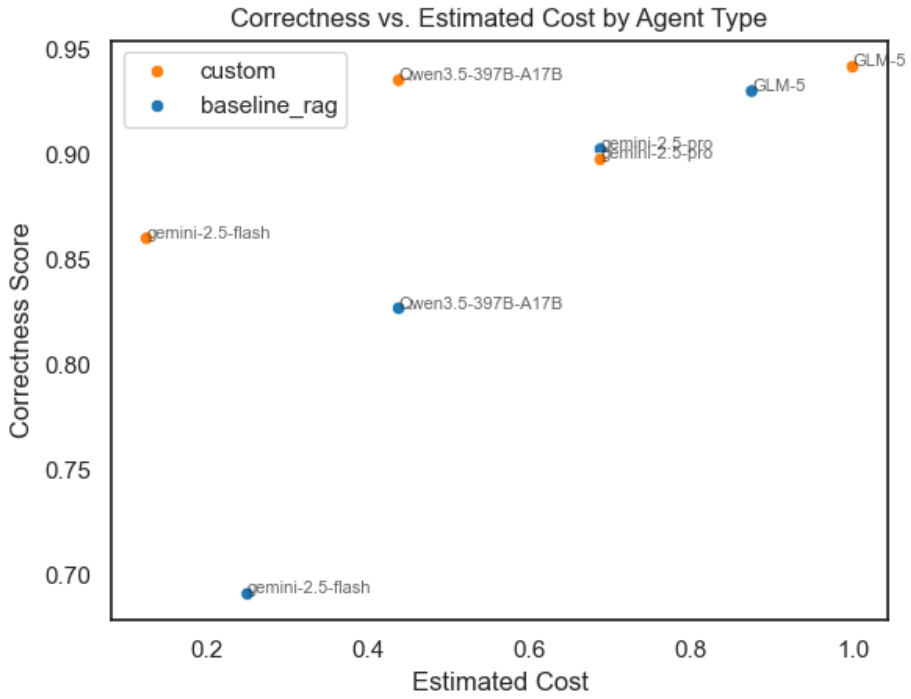

Figure 4.13: Estimated relative cost rank vs. correctness score per query. The custom agent (orange) occupies the upper-left region for several models, achieving higher correctness at lower or equal relative cost.

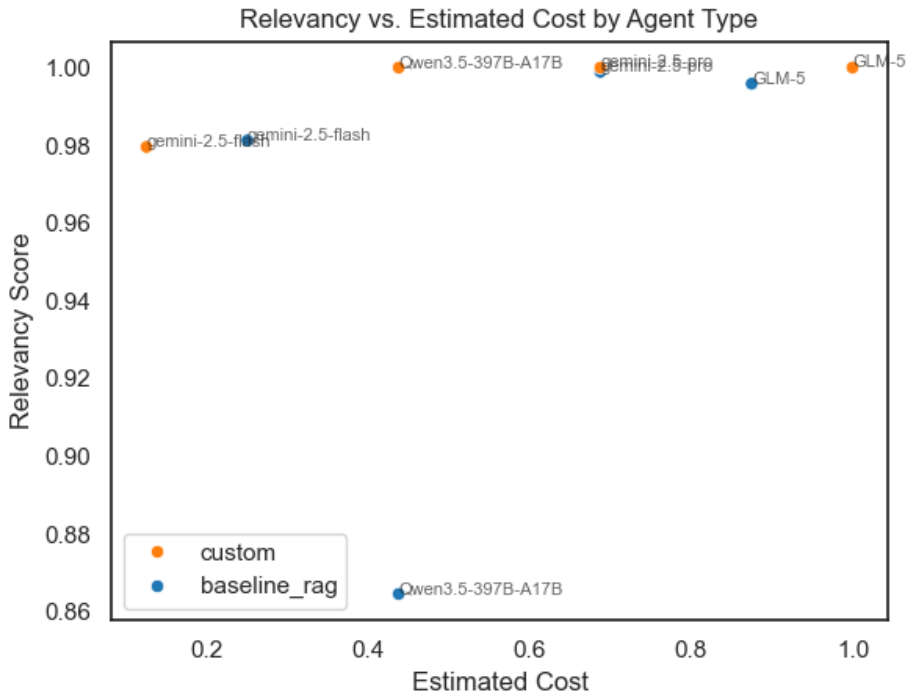

Figure 4.14: Estimated relative cost rank vs relevancy score per query. Relevancy is less sensitive to architecture than correctness: both configurations cluster near the top of the relevancy scale regardless of cost, reflecting that on-topic responses are easy to achieve once retrieval succeeds.

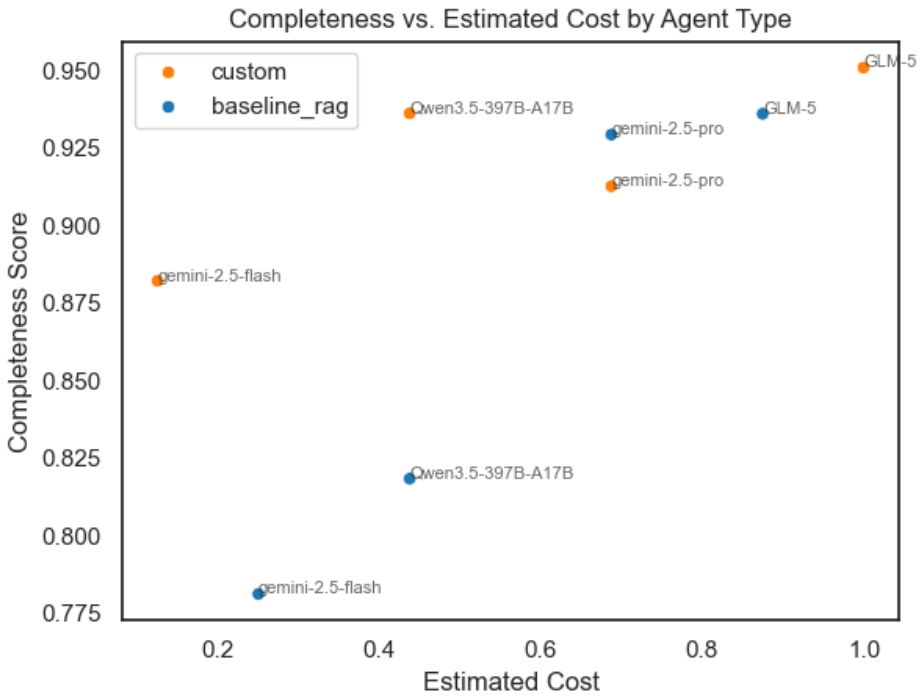

Figure 4.15: Estimated relative cost rank vs. completeness score per query. The custom agent dominates the upper portion of the completeness axis at comparable or lower cost, mirroring the correctness pattern.

The correctness and completeness scatter plots reveal a consistent pattern: the custom agent (orange) occupies the upper portion of both axes at comparable or lower relative cost for most models. The most striking case is `gemini-2.5-flash`, where the custom configuration achieves a cost rank of 0.125 (cheapest in the set) while delivering a correctness score of 0.860, versus the baseline's rank of 0.250 and score of 0.691. Relevancy, by contrast, is less discriminative: both configurations cluster near 1.0 regardless of cost, consistent with the earlier finding that on-topic responses are easy to achieve once retrieval succeeds at all.

Table 4.4 summarises mean cost rank and quality scores per model and agent configuration.

Table 4.4: Mean relative cost rank and quality scores per model and agent configuration. Cost rank is a within-dataset percentile (0 = cheapest, 1 = most expensive).

Model	Agent	Cost rank	Correctness	Relevancy	Completeness
GLM-5	baseline_rag	0.875	0.930	0.996	0.936
GLM-5	custom	1.000	0.942	1.000	0.951
Qwen3.5-397B	baseline_rag	0.438	0.827	0.865	0.818
Qwen3.5-397B	custom	0.438	0.935	1.000	0.936
gemini-2.5-flash	baseline_rag	0.250	0.691	0.981	0.781
gemini-2.5-flash	custom	0.125	0.860	0.980	0.882
gemini-2.5-pro	baseline_rag	0.688	0.902	0.999	0.929
gemini-2.5-pro	custom	0.688	0.898	1.000	0.913

The table highlights the practical trade-off. For three of the four models tested, the custom agent achieves equal or lower cost rank while delivering meaningfully higher correctness and completeness. `gemini-2.5-pro` is the exception: both configurations share the same cost rank and their quality scores are within one percentage point, confirming that the most capable model compensates for the absence of structured context through its own reasoning. The structured factsheet therefore functions most effectively as a cost-efficient capability amplifier for mid-tier models.

Actual vs. Advertised Cost. A further nuance emerges when examining the relationship between advertised input token price and the actual, usage-weighted input cost. The advertised price per token does not fully predict the total cost of a query, because more capable models may use tools more efficiently, requiring fewer tokens overall to arrive at an answer. Figure 4.16 illustrates this by plotting the ranked actual input cost against the advertised price.

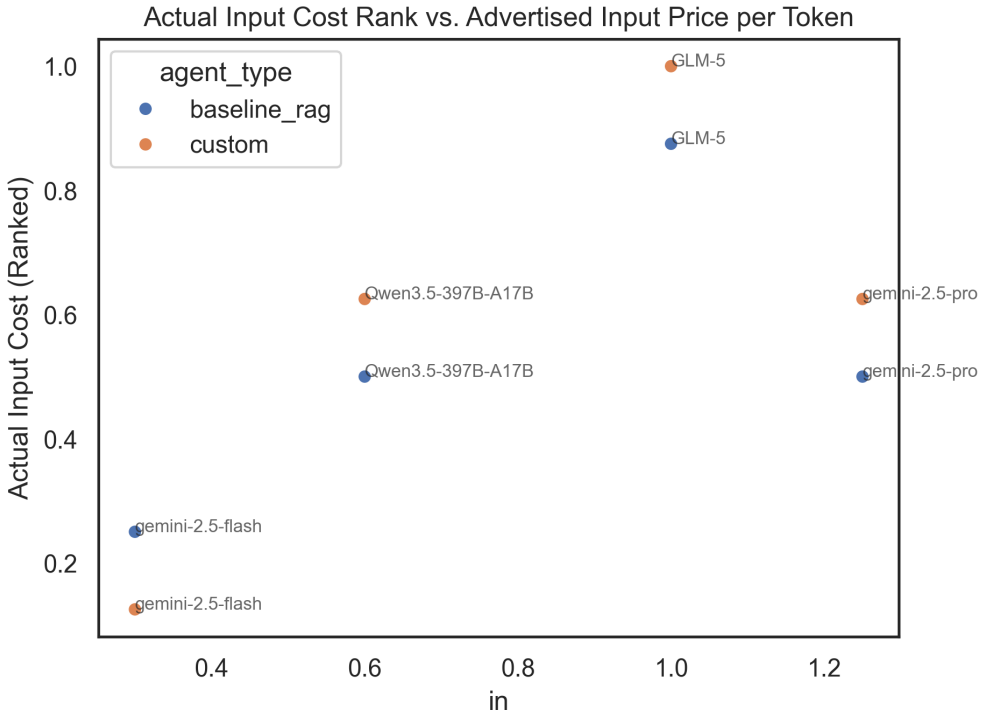

Figure 4.16: Actual input cost (ranked) vs. advertised input price per token. The relationship is non-linear, indicating that more capable models can be more token-efficient.

The relationship is notably non-linear. The open-source models, despite having competitive advertised prices, tend to incur a higher relative `actual_input_cost_rank`. Conversely, the closed-source models demonstrate greater token efficiency. `gemini-2.5-pro`, for instance, shows a significant drop in actual cost rank compared to its position on the price axis, performing on par with models that are nominally cheaper per token. `gemini-2.5-flash` also performs well on this metric, delivering a low actual cost. This finding, while secondary to the main research questions, is a valuable insight for practical deployment: the true cost of operating a model is a function of both its price and its underlying capability to reason efficiently and use tools judiciously. The custom agent architecture does not significantly alter this dynamic, as its primary contribution is to the quality and consistency of the context, not the model's intrinsic reasoning efficiency.

4.2 Statistical Modeling

4.2.1 Mann-Whitney U Test

Hypothesis tests follow the design described in Section 3.3.4. Normality was confirmed to be absent via Shapiro-Wilk ($p < 10^{-18}$ for every metric tested), validating the use of the one-sided Mann-Whitney U test as described in Chapter 3.

Quality metrics. Table 4.5 summarises the results. Correctness and relevancy both show statistically significant advantages for the custom agent at the $\alpha = 0.05$ level. Completeness does not reach significance, though the directional trend is the same.

Table 4.5: Mann-Whitney U test results for quality metrics (one-sided, H_1: custom > baseline).

Metric	U-statistic	p-value	Result
Correctness	74,780.50	1.03×10^{-2}	Significant ($p < 0.05$)
Relevancy	72,733.00	5.54×10^{-5}	Significant ($p < 0.05$)
Completeness	27,278.50	1.06×10^{-1}	Not significant

The relevancy result warrants a cautious interpretation. Relevancy is almost universally high for both agents whenever a response is attempted: as long as the agent engages with the question, the answer tends to be on-topic regardless of architecture. The significant difference is primarily driven by cases where the baseline fails to retrieve any relevant documents and consequently produces no substantive answer, incurring a near-zero relevancy score. **Correctness is the most diagnostic quality metric**: it is hardest to satisfy and most sensitive to the architectural differences under evaluation, and the significant result ($p = 1.03 \times 10^{-2}$) carries the most interpretive weight.

The absence of a significant completeness result likely reflects the smaller number of completeness observations in the dataset and the inherent difficulty of distinguishing partial from complete answers when both agents frequently cover the required facts.

Resource metrics. Neither input token usage (U = 71,961, p = 0.859) nor inference duration (U = 69,100, p = 0.538) differs significantly between configurations. Mean input token counts are comparable (custom: 79,319 ± 82,405; baseline: 75,395 ± 78,670) and mean durations are within one second of each other (custom: 39.8 s ± 27.2 s; baseline: 39.6 s ± 28.9 s). The custom agent achieves its quality improvements without imposing a measurable additional cost in tokens or latency.

4.2.2 Levene's Test for Equality of Variance

While the Mann-Whitney U test establishes that the custom agent's scores tend to be stochastically *higher*, it says nothing about the *spread* of those scores. Levene's test for equality of variance addresses this directly: it tests whether the two configurations exhibit significantly different score variability, without assuming normality. The results are presented in Table 4.6.

Table 4.6: Levene's test results for equality of variance (custom vs. baseline+RAG). A significant result ($p < 0.05$) indicates the two configurations differ in score variability.

Metric	Levene statistic	p-value	Result
Correctness	18.38	< 0.001	Significant
Relevancy	11.56	0.0007	Significant
Completeness	7.76	0.0056	Significant
BERTScore F1	7.41	0.0066	Significant
Sentence-BERT sim.	16.13	< 0.001	Significant
Duration (s)	0.25	0.620	Not significant
Input tokens	0.02	0.885	Not significant
Self-consistency	0.06	0.810	Not significant

The test returns a significant result for all five quality metrics, including the complementary BERTScore and sentence-BERT similarity where the mean difference is small. This means the custom agent is not only more accurate on average — it is *more consistent*. The baseline's distribution is significantly wider across all quality dimensions, with a heavier low-scoring tail representing cases of partial or failed retrieval. The custom agent's reduced variance implies more predictable, reliable responses: fewer failures at the low end and less dependence on whether retrieval happens to surface the right chunks for a given query.

By contrast, variance in inference duration, input token counts, and self-consistency scores does not differ significantly between configurations. The resource overhead of the two agents is therefore comparable not only in expectation but also in spread, and self-consistency variability is similarly matched. The reliability advantage of the custom agent is specific to output quality, not a general property of all metrics.

In a legal setting, this reliability dimension may be as practically significant as the mean improvement. A system that occasionally fails unpredictably is harder to trust and supervise than one that performs steadily, even if its average score is similar.

4.2.3 Self-consistency

Figure 4.17: Distribution of self-consistency scores by agent type. The custom agent shows a higher median and narrower interquartile range.

Self-consistency scores reinforce the pattern seen in the DeepEval metrics visually: the custom agent exhibits a higher median and a narrower interquartile range than the baseline, suggesting more reproducible responses across sessions. However, a one-sided Mann-Whitney U test does not confirm a statistically significant advantage ($U = 7{,}039$, $p = 0.875$). Similarly, Levene's test finds no significant difference in self-consistency variance between the two configurations (statistic $= 0.06$, $p = 0.810$). The descriptive pattern is therefore directionally consistent with the quality findings but does not reach significance, likely due to the smaller number of observations available at the session-group level.

4.3 Summary by Research Question

Table 4.7 summarises the empirical evidence for each research question.

Table 4.7: Research question outcomes based on empirical results.

RQ	Finding	Evidence
Main	Custom agent improves factual accuracy without significant cost increase	Correctness significant (p=1.03×10^{-2}); token usage and latency statistically equivalent
RQ1	Evidence is directionally consistent but inconclusive; a larger gap on the more complex dataset is suggestive but rests on few sessions and does not hold uniformly	Wider gap on TOSL-2024 sessions 3–4 only; THRD-2021 shows no clear difference
RQ2	Custom agent maintains stable or improving quality across sessions; baseline performance is more variable and lower throughout	Session trajectory plots; narrower confidence intervals and higher median for custom agent at later sessions
RQ3	Structured context amplifies smaller model performance; the gap between custom and baseline agents narrows as model capability increases	LLM barplot: gap is widest for smaller models, negligible for the most capable model

Main RQ. The results support an affirmative answer: the custom agent achieves statistically significant improvements in correctness and relevancy over the RAG-only baseline, while consuming a statistically equivalent number of tokens and producing responses within the same latency envelope. The quality gain is not purchased at a proportional resource cost.

RQ1. The evidence for RQ1 is directionally consistent with the hypothesis but not strong enough to support a firm conclusion. THRD-2021 shows no clear performance gap between configurations. TOSL-2024 shows a somewhat wider gap in sessions 3–4, but this rests on a small number of sessions and does not hold uniformly across all metrics. The data is too thin to isolate whether the effect is driven by cross-document synthesis demands specifically or by other case-level differences in complexity and document type. RQ1 remains open and would require a targeted benchmark of explicitly synthesis-demanding queries to resolve conclusively.

RQ2. Session-level trajectories show that the custom agent maintains quality across all sessions with narrower variance, while the baseline exhibits wider swings and a less stable profile. This supports the conclusion that the structured representation scales with document volume rather than degrading under it.

RQ3. The model-level breakdown provides the clearest evidence for RQ3: the custom agent's structured context delivers the largest marginal gain for smaller and less capable models, with the gap shrinking to near-zero for the most advanced model tested. This suggests that the factsheet acts as a capability compensator — partially substituting for the reasoning capacity that larger models supply natively.

Chapter 5

Discussion

5.1 Research Question Findings

This section interprets the empirical results in relation to each research question, drawing on the statistical evidence from Chapter 4 and situating the findings within the theoretical context established in Chapter 2.

Main RQ: accuracy improvement without proportional cost. The results support an affirmative answer. The custom agent achieves statistically significant improvements in correctness ($p = 8.5 \times 10^{-3}$) and relevancy ($p = 3.4 \times 10^{-4}$) while consuming a statistically equivalent number of tokens and inference time compared to the RAG-only baseline. The context compression mechanisms — significance filtering and display shortening applied server-side before context injection — are largely responsible for keeping token usage in check as the factsheet grows: rather than injecting the full structured state verbatim, the agent receives a filtered, abbreviated representation tailored to the query context. The combined picture is that schema-driven extraction improves factual accuracy at no measurable additional cost per query.

Beyond the mean improvement, the variance reduction is arguably the most consistent finding in the evaluation. Levene's test for equality of variance is significant across all five metrics tested — correctness, relevancy, completeness, BERTScore F1, and sentence-BERT similarity — including the complementary metrics where the mean difference is modest. The baseline agent's score distributions are significantly wider, with a heavier low-scoring tail that reflects the brittleness of pure retrieval: when the relevant chunks do not rank highly for a given query, performance drops sharply. The custom agent's structured state provides a reliable fallback that absorbs these retrieval failures, yielding more predictable responses across the board. In a professional legal setting, this consistency advantage may carry more practical weight than the mean improvement alone: a system that occasionally fails unpredictably is harder to trust and integrate into a practitioner's workflow than one that performs steadily.

RQ1: mitigation of cross-document retrieval limitations. The evidence for RQ1 is suggestive but not conclusive, and the conclusion should be stated carefully. THRD-2021 shows no clear performance gap between the two architectures. TOSL-2024, the larger and more complex authentic case, shows a somewhat more pronounced advantage for the custom agent in the later sessions (sessions 3–4), but this pattern rests on a small number of sessions and does not hold uniformly across all metrics. With only two datasets and relatively few sessions each, the data is too thin to isolate whether the difference reflects document complexity, query-type composition, or genuine cross-document synthesis demands.

A further theoretical nuance is worth noting: naive RAG is not straightforwardly *volume-sensitive* in the way the original framing implies. Because retrieval always returns a fixed top-K regardless of corpus size, a larger document pool does not mechanically degrade retrieval quality. What RAG genuinely struggles with is *synthesis queries* — questions whose answers are distributed across multiple chunks that cannot all fit within a single top-K result. This is a query-type limitation, not a volume limitation. The evaluation did not explicitly control for the proportion of synthesis queries across

sessions and datasets, making it difficult to attribute the observed TOSL-2024 pattern specifically to cross-document retrieval failure rather than to other case-specific factors.

In summary, the results are directionally consistent with the hypothesis that structured extraction mitigates cross-document retrieval limitations, but the empirical basis is insufficient to support a strong conclusion. Confirming RQ1 would require a larger benchmark with sessions explicitly designed around synthesis-demanding queries.

RQ2: stability across growing document volume. Session-level trajectories show that the custom agent maintains consistent quality with narrow variance throughout case progression, while the baseline exhibits wider fluctuations and a less stable profile as sessions accumulate new documents. This is consistent with the design hypothesis: the factsheet absorbs new information incrementally at each session boundary, so the agent's knowledge of the case becomes richer rather than more fragmented as volume grows. The baseline, by contrast, must retrieve from an ever-larger document pool with no persistent structure to guide retrieval, making its performance increasingly dependent on whether the relevant chunks happen to rank highly in a given semantic search.

RQ3: compensation for reduced model capacity. The model-level breakdown provides the most striking result for RQ3: the performance gap between the custom and baseline agents is largest for the smaller models and shrinks to near-zero for the most capable model tested. For smaller models, the structured factsheet provides the key facts in an organised, pre-digested form, reducing the reasoning burden on the model itself. A more capable model can compensate for noisy or incomplete retrieval through superior inference, making the factsheet less decisive. This model-dependent pattern has a direct practical implication for cost management: a smaller, less expensive model paired with the custom architecture can approximate the performance of a larger, more expensive model on the RAG-only baseline, suggesting a cost-quality substitution that may be valuable in production deployments.

5.2 Value Beyond Performance Metrics

The evaluation results demonstrate a performance advantage for the custom architecture as case volume grows, but the headline metrics alone understate the practical value of the design. Even in a hypothetical scenario where correctness and completeness scores were identical across configurations, the custom architecture would remain preferable in a production legal setting for reasons that no automatic metric captures.

The factsheet as a product artifact. The baseline agent's output is ephemeral: it produces a text response and leaves no persistent structured representation of the case behind. The custom agent, by contrast, produces and continuously maintains a structured factsheet as a natural side-effect of normal operation. This factsheet—comprising parties, events, claims, damages, deadlines, and document metadata—is a machine-readable, queryable representation of the case that can be consumed directly by frontend systems without any additional LLM inference. Concrete applications include timeline visualisations, party graphs, claims dashboards, and deadline trackers, all of which can be rendered from the factsheet with no extra cost.

Human oversight and auditability. The structured state enables a qualitatively different relationship between the lawyer and the system. Rather than trusting a black-box text response, a practitioner can inspect the factsheet directly, verify individual entries against source documents, and correct errors at the element level. If the agent produces an incorrect answer, the source of the error can often be traced to a specific factsheet entry—a mis-extracted event date or an omitted party—and corrected there, improving all subsequent responses. This kind of targeted, human-in-the-loop correction is not possible with a stateless RAG agent, where errors are buried in the implicit context and must be addressed by re-uploading or re-prompting.

Versioned case understanding. Because the factsheet is updated incrementally at each session boundary, it constitutes a natural audit trail of how the agent's understanding of the case evolved over time. Earlier versions can be retained and compared, providing a record of when specific facts, parties,

or claims entered the agent's knowledge base. In legal practice, where the timing of knowledge can itself be legally significant, this version history has potential evidentiary and procedural value.

Downstream system integration. The structured state lowers the integration barrier for adjacent legal tools and workflows. A downstream system can query the factsheet via a standard database interface to, for example, check for conflicts of interest against other open cases, trigger calendar entries for extracted deadlines, or populate court filing templates with party and claim data. These integrations require structured, typed data; they cannot be built on top of free-form agent responses without a costly and fragile natural language parsing layer.

Summary. The performance metrics reported in Chapter 4 capture one dimension of the architecture's value: how accurately the agent answers factual questions across a progression of sessions. They do not capture the value of the structured case representation itself—a persistent, inspectable, and reusable artifact that the custom architecture produces as a consequence of its design. In production deployments with real case volumes and real practitioner workflows, this latent value is likely to be at least as significant as the raw performance advantage.

5.3 Uncertainties and Confounding Factors

The evaluation results support the conclusion that the custom agent outperforms the baseline on correctness and relevancy, but several sources of uncertainty affect the magnitude and generalisability of this finding. These are discussed below.

Question difficulty and scope. The evaluation queries were designed to be factual and document-grounded, but the level of specificity varies considerably across sessions. Some questions have unambiguous single-document answers, while others require synthesising information across multiple documents and timelines. Harder queries amplify the architectural advantage of the custom agent — the factsheet and structured retrieval tools are most useful precisely when the answer requires cross-document reasoning — while easier queries may show little to no difference between architectures. The observed aggregate performance gap therefore depends on the composition of the query set: a test set dominated by simple lookups would understate the custom agent's advantage, while one focused on complex synthesis would overstate it. The current evaluation mix likely sits somewhere in between.

Dataset size and complexity. Only two datasets are used in the evaluation, and their characteristics differ substantially. The results from TOSL-2024, the larger and more complex real case, show a more pronounced advantage for the custom agent than those from THRD-2021. This is consistent with the design hypothesis: the factsheet-based architecture is most beneficial when the document set is large and the case is complex, because those are the conditions under which naive retrieval is most likely to fail. With only two datasets, however, it is difficult to disentangle the effect of document volume from other case-specific characteristics. A larger benchmark spanning more cases and varying document volumes would provide stronger evidence for the scaling behaviour observed here.

Underlying language model capability. Figure 4.7 shows that the performance gap between the custom and baseline agents narrows as the underlying LLM becomes more capable: for the most advanced model tested (Gemini 2.5 Pro), the two configurations are nearly indistinguishable. This suggests two non-exclusive interpretations. First, a sufficiently capable model may compensate for the structural disadvantage of the baseline by inferring missing context or tolerating retrieval noise more robustly. Second, the evaluation queries and datasets may not be sufficiently demanding to expose the limits of a powerful model: the questions can largely be answered from the retrieved chunks even without the accumulated factsheet context. Conversely, the performance gap is most pronounced for the smaller models, for which structured, pre-digested context provides the greatest marginal benefit. This implies that the value of the custom architecture is not fixed but model-dependent: it acts as a capability amplifier that matters most when the underlying model has limited reasoning capacity or context-handling ability.

Stochastic variability. LLM outputs are inherently stochastic. Each evaluation set was run three times to reduce the impact of this variability, and the non-parametric Mann-Whitney U test is robust to the non-normal, left-skewed score distributions observed. Nevertheless, the total number of unique query-session combinations is constrained by the cost and time of running large-scale multi-model evaluations, and the reported p-values should be interpreted with awareness of the limited sample size. The significance of correctness ($p = 8.5 \times 10^{-3}$) and relevancy ($p = 3.4 \times 10^{-4}$) results are robust, but the borderline result for completeness ($p = 8.3 \times 10^{-2}$) should be treated cautiously.

Chapter 6

Conclusion

This thesis investigated whether schema-driven cross-document extraction into a structured case representation — a *FactSheet* — improves the factual accuracy of an agentic RAG system in multi-document legal settings, and whether it does so without incurring proportional increases in query-time cost. The motivation was grounded in a fundamental limitation of chunk-based retrieval: when the answer to a legal query is distributed across multiple documents, no amount of semantic similarity tuning can make an isolated-chunk retriever assemble a coherent synthesis. The proposed approach addresses this by extracting typed entities — parties, events, claims, damages, deadlines — from all case documents at ingestion time and maintaining them as a persistent, relational representation that supplements retrieval for case-level queries.

6.1 Answers to Research Questions

Main RQ. Schema-driven extraction into a structured case representation does improve factual accuracy: the custom agent achieves statistically significant improvements in correctness ($p = 8.5 \times 10^{-3}$) and relevancy ($p = 3.4 \times 10^{-4}$) relative to the RAG-only baseline. Token consumption and inference latency are statistically equivalent between the two architectures, confirming that the quality gain is not purchased at a proportional resource cost. The main research question is answered affirmatively.

RQ1. The evidence for RQ1 is directionally consistent with the hypothesis but insufficient for a firm conclusion. THRD-2021 shows no clear performance gap between configurations. TOSL-2024 shows a somewhat wider advantage for the custom agent in the later sessions, but this rests on few sessions and does not hold uniformly across all metrics. A further theoretical nuance applies: naive RAG is not straightforwardly volume-sensitive, since retrieval always returns a fixed top-K regardless of corpus size. The genuine limitation is query-type specific — synthesis queries whose answers are distributed across multiple chunks — and the evaluation did not explicitly control for this dimension. RQ1 therefore remains open; confirming it would require a dedicated benchmark of synthesis-demanding queries across a larger number of sessions and cases.

RQ2. Session-level trajectories show that the custom agent maintains stable quality with narrow variance across all sessions as document volume grows. The baseline, by contrast, exhibits wider fluctuations throughout case progression. The structured representation scales gracefully with document volume because new information is absorbed incrementally into the factsheet at each session boundary rather than added to an ever-larger unstructured retrieval pool. RQ2 is answered affirmatively.

RQ3. The model-level breakdown reveals that the performance gap between the custom and baseline agents is largest for smaller, less capable models and narrows to near-zero for the most advanced model tested. Structured, pre-digested context reduces the reasoning burden placed on the underlying model, enabling smaller models to approximate the performance of larger ones on the RAG-only baseline. This constitutes a practically significant cost-quality substitution: a cheaper model paired with the custom architecture can match a more expensive model operating on unstructured retrieval alone. RQ3 is

answered affirmatively, with the caveat that the effect is model-dependent and diminishes as base model capability increases.

6.2 Additional Finding: Variance Reduction

Beyond the hypothesised mean improvement, the evaluation reveals a consistent and statistically robust reduction in score variance for the custom agent. Levene's test for equality of variance is significant across all five metrics tested — correctness, relevancy, completeness, BERTScore F1, and sentence-BERT similarity — including the complementary metrics where mean differences are modest. This finding was not pre-specified as a research question; it emerged from the distributional analysis of the results.

The interpretation is straightforward: pure retrieval is brittle. When relevant chunks happen not to rank highly for a given query, the baseline agent's performance drops sharply, producing the heavy low-scoring tail visible across all its metric distributions. The custom agent's structured state provides a stable knowledge base that does not depend on retrieval succeeding for any individual query, dampening these failures and yielding more predictable responses.

In a legal deployment context, this reliability dimension may carry more practical weight than the mean improvement. A practitioner integrating an AI assistant into their workflow needs to be able to predict when the system will and will not perform well. A system with lower variance is easier to trust, easier to supervise, and easier to integrate into structured legal processes than one with comparable average accuracy but unpredictable failure modes.

6.3 Contributions

This thesis makes three contributions:

1. **System design.** A schema-driven initialization and incremental update pipeline for structured legal case state management, implemented as a LangGraph `StateGraph` with fan-out parallel processing, PostgreSQL relational storage, and configurable context compression.

2. **Evaluation framework.** A multi-model evaluation setup comparing a structured-state agent against a RAG-only baseline across two legal datasets, multiple session stages, and three repeated runs per configuration, using a combination of semantic quality metrics (DeepEval correctness, relevancy, completeness), surface similarity metrics (BERTScore), consistency metrics, and resource consumption measures.

3. **Empirical evidence.** Quantitative evidence that structured extraction improves factual accuracy in multi-document legal settings without proportional cost increase, with the effect being strongest on complex cases, stable across session progression, and most pronounced for smaller models.

6.4 Limitations and Future Work

The evaluation in this thesis has several limitations that point toward avenues for future research.

Dataset and Generalisability. The evaluation rests on two datasets, which limits the generalisability of the findings. A larger benchmark spanning more cases and document volumes would provide stronger evidence for the scaling behaviour observed here, and would better distinguish the effect of document volume from other case-specific characteristics. A potential weakness is that the datasets were developed in conjunction with this thesis, which introduces a risk that their design was implicitly influenced by the research questions, potentially limiting their independence as a neutral benchmark.

A more fundamental limitation, however, is the scarcity of publicly available, high-quality datasets suitable for benchmarking legal agents in realistic, long-duration scenarios. The datasets used, while adequate for the research questions posed, represent a simplified proxy for the complexity of a genuine legal case that unfolds over months or years. The evaluation was necessarily limited to a sequence of sessions within a static, pre-existing document corpus. A crucial direction for future work is therefore

the development of high-quality, publicly available datasets designed specifically for full-scale, real-world scenario testing. Such benchmarks are essential for the field to move beyond simple, short-form question-answering formats and begin to test agent capabilities in a manner that mirrors actual legal practice. These datasets should be designed to test agents across time, presenting new information and evolving case dynamics in a structured way that challenges an agent's ability to maintain a coherent, long-term understanding of a case. Creating such a benchmark is a substantial research project in its own right, arguably the subject of a separate thesis. It would require careful curation of case materials, the design of a longitudinal evaluation protocol, and the definition of tasks that probe for deep synthesis and long-range reasoning rather than simple fact retrieval.

Scope of Legal Reasoning. The evaluation queries were deliberately restricted to factual, document-grounded questions to enable objective grading. This scope is appropriate for establishing a controlled baseline but does not cover the full range of legal reasoning tasks — in particular, normative and predictive questions where the benefit of structured extraction may differ. Extending the evaluation to cover multi-step legal reasoning, conflicting evidence reconciliation, and temporal argument construction is a natural next step.

Schema Flexibility. The structured representation in this thesis uses a fixed entity schema (parties, events, claims, damages, deadlines). For cases with different legal structures, such as regulatory compliance matters or transactional disputes, the schema may require adaptation. An interesting direction is schema induction: learning the relevant entity types from the case documents themselves rather than relying on a predefined legal ontology.

Cost-Quality Optimisation. The RQ3 result points toward a practically relevant optimization problem: given a target quality level and a budget constraint, what is the optimal combination of model size and extraction investment? The evidence suggests this trade-off exists and is exploitable, but characterising it precisely requires a more systematic sweep over model and architecture configurations than was feasible within the scope of this thesis.

Reliability as a First-Class Research Question. The variance reduction observed across all five metrics is a post-hoc finding and was not designed into the evaluation. A natural follow-up study would treat reliability as a first-class research question: formally hypothesising that structured extraction reduces output variance, designing the evaluation around variance-sensitive query sets (particularly retrieval-failure-prone queries), and testing across a broader range of cases and document volumes. Given the practical significance of predictability in legal settings, this direction warrants dedicated investigation.

Advanced Retrieval over Structured Elements. The current implementation queries the fact-sheet through simple SQL lookups filtered by date range and significance level. The richness of the extracted representation — typed entities with source references, timestamps, significance scores, and relational links — supports substantially more sophisticated retrieval strategies. Graph traversal across entity relationships, cross-referencing events with the documents that substantiate them, and ranking elements by their relevance to a specific query rather than by static significance are all unexplored. The factsheet as implemented is a data foundation whose retrieval layer has not been fully exploited.

Agentic Reasoning and Task Queuing. The current agent executes a single reasoning pass per query. Adding a task queue that allows the agent to decompose complex queries into ordered sub-tasks — each potentially using a specialised node, toolset, and system prompt — would support multi-step legal reasoning that the current flat architecture cannot perform. This is analogous to chain-of-thought planning at the agent level: rather than producing an answer in one shot, the agent could plan, retrieve, verify, and synthesise across multiple coordinated steps.

Background and Asynchronous Tooling. The pipeline already runs offline, but the agent itself is purely reactive: it only acts when a query arrives. A natural extension is a set of background tools that run during idle periods — monitoring connected document repositories and inboxes for new material,

triggering incremental factsheet updates on arrival, or proactively flagging newly detected deadlines and conflicts to the practitioner. This would move the system from reactive assistant to proactive case monitor.

Graph Database and GraphRAG. The relational PostgreSQL schema captures entity types and source links but does not natively represent or traverse relationships between entities. Migrating the structured representation to a graph database would enable direct traversal of entity relationships — for example, following a chain from a claim to the events that substantiate it to the documents that evidence those events. Combined with graph-native retrieval, this would address the cross-document reasoning limitation more completely than the current hybrid approach and bring the system closer to a full domain-specific GraphRAG architecture.

Bibliography

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.

[2] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. Ruler: What's the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024.

[3] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts, 2023.

[4] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2021.

[5] Yunfan Gao, Yun Xiong, Yijie Zhong, Yuxi Bi, Ming Xue, and Haofen Wang. Synergizing rag and reasoning: A systematic review, 2025.

[6] Yixuan Tang and Yi Yang. Multihop-RAG: Benchmarking retrieval-augmented generation for multi-hop queries. In *First Conference on Language Modeling*, 2024.

[7] Jintao Liang, Gang Su, Huifeng Lin, You Wu, Rui Zhao, and Ziyue Li. Reasoning rag via system 1 or system 2: A survey on reasoning agentic retrieval-augmented generation for industry challenges, 2025.

[8] Pengcheng Jiang, Siru Ouyang, Yizhu Jiao, Ming Zhong, Runchu Tian, and Jiawei Han. Retrieval and structuring augmented generation with large language models. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*, KDD '25, page 6032–6042. ACM, August 2025.

[9] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph rag approach to query-focused summarization, 2025.

[10] Hao Liu, Zhengren Wang, Xi Chen, Zhiyu Li, Feiyu Xiong, Qinhan Yu, and Wentao Zhang. Hoprag: Multi-hop reasoning for logic-aware retrieval-augmented generation. *arXiv preprint arXiv:2502.12442*, 2025.

[11] Microsoft. Semantic kernel python: Context management, 2024. Accessed: 2025-11-26.

[12] LangChain. Langgraph, 2024. Accessed: 2025-11-26.

[13] Ilias Chalkidis, Manos Fergadiotis, Prodromos Malakasiotis, Nikolaos Aletras, and Ion Androutsopoulos. LEGAL-BERT: The muppets straight out of law school. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 2898–2904. Association for Computational Linguistics, 2020.

[14] Neel Guha, Julian Nyarko, Daniel E. Ho, Christopher Ré, Adam Chilton, Aditya Narayana, Alex Chohlas-Wood, Austin Peters, Brandon Waldon, Daniel N. Rockmore, Diego Zambrano, Dmitry Talisman, Enam Hoque, Faiz Surani, Frank Fagan, Galit Sarfaty, Gregory M. Dickinson, Haggai Porat, Jason Hegland, Jessica Wu, Joe Nudell, Joel Niklaus, John Nay, Jonathan H. Choi, Kevin

Tobia, Margaret Hagan, Megan Ma, Michael Livermore, Nikon Rasumov-Rahe, Nils Holzenberger, Noam Kolt, Peter Henderson, Sean Rehaag, Sharad Goel, Shang Gao, Spencer Williams, Sunny Gandhi, Tom Zur, Varun Iyer, and Zehua Li. LegalBench: A collaboratively built benchmark for measuring legal reasoning in large language models, 2023.

[15] Antoine Louis, Gijs van Dijck, and Gerasimos Spanakis. Interpretable long-form legal question answering with retrieval-augmented large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, number 20, pages 22266–22275, 2024.

[16] Supabase. Supabase, 2020.

[17] Sebastián Ramírez. Fastapi, 2018. Accessed: 2025-11-26.

[18] Omid Shahmirzadi, Zhaofeng Wu, Robert T. Logan IV, Jonathan K. Kummerfeld, and Rada Mihalcea. Multi-lexsum: A multi-document summarization dataset for the legal domain, 2025.

[19] Yixuan Li, Wei Ye, Ziyan Wang, Yujia Li, and Shikun Zhang. Casegen: A large language model for generating case-based questions in legal scenarios, 2025.

[20] Chengyu Fang and Ashish Kulkarni. Enronqa: A q&a dataset for private document collections, 2023.

[21] Confident AI. Deepeval: The llm evaluation framework, 2023.

[22] Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026.

[23] Together AI. Together ai.
urlhttps://together.ai/, 2024. Accessed: 2026-04-08.

[24] Qwen Team. Qwen3 technical report, 2025.

[25] GLM-5-Team, :, Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, Chengxing Xie, Chenzheng Zhu, Congfeng Yin, Cunxiang Wang, Gengzheng Pan, Hao Zeng, Haoke Zhang, Haoran Wang, Huilong Chen, Jiajie Zhang, Jian Jiao, Jiaqi Guo, Jingsen Wang, Jingzhao Du, Jinzhu Wu, Kedong Wang, Lei Li, Lin Fan, Lucen Zhong, Mingdao Liu, Mingming Zhao, Pengfan Du, Qian Dong, Rui Lu, Shuang-Li, Shulin Cao, Song Liu, Ting Jiang, Xiaodong Chen, Xiaohan Zhang, Xuancheng Huang, Xuezhen Dong, Yabo Xu, Yao Wei, Yifan An, Yilin Niu, Yitong Zhu, Yuanhao Wen, Yukuo Cen, Yushi Bai, Zhongpei Qiao, Zihan Wang, Zikang Wang, Zilin Zhu, Ziqiang Liu, Zixuan Li, Bojie Wang, Bosi Wen, Can Huang, Changpeng Cai, Chao Yu, Chen Li, Chengwei Hu, Chenhui Zhang, Dan Zhang, Daoyan Lin, Dayong Yang, Di Wang, Ding Ai, Erle Zhu, Fangzhou Yi, Feiyu Chen, Guohong Wen, Hailong Sun, Haisha Zhao, Haiyi Hu, Hanchen Zhang, Hanrui Liu, Hanyu Zhang, Hao Peng, Hao Tai, Haobo Zhang, He Liu, Hongwei Wang, Hongxi Yan, Hongyu Ge, Huan Liu, Huanpeng Chu, Jia'ni Zhao, Jiachen Wang, Jiajing Zhao, Jiamin Ren, Jiapeng Wang, Jiaxin Zhang, Jiayi Gui, Jiayue Zhao, Jijie Li, Jing An, Jing Li, Jingwei Yuan, Jinhua Du, Jinxin Liu, Junkai Zhi, Junwen Duan, Kaiyue Zhou, Kangjian Wei, Ke Wang, Keyun Luo, Laiqiang Zhang, Leigang Sha, Liang Xu, Lindong Wu, Lintao Ding, Lu Chen, Minghao Li, Nianyi Lin, Pan Ta, Qiang Zou, Rongjun Song, Ruiqi Yang, Shangqing Tu, Shangtong Yang, Shaoxiang Wu, Shengyan Zhang, Shijie Li, Shuang Li, Shuyi Fan, Wei Qin, Wei Tian, Weining Zhang, Wenbo Yu, Wenjie Liang, Xiang Kuang, Xiangmeng Cheng, Xiangyang Li, Xiaoquan Yan, Xiaowei Hu, Xiaoying Ling, Xing Fan, Xingye Xia, Xinyuan Zhang, Xinze Zhang, Xirui Pan, Xu Zou, Xunkai Zhang, Yadi Liu, Yandong Wu, Yanfu Li, Yidong Wang, Yifan Zhu, Yijun Tan, Yilin Zhou, Yiming Pan, Ying Zhang, Yinpei Su, Yipeng Geng, Yong Yan, Yonglin Tan, Yuean Bi, Yuhan Shen, Yuhao Yang, Yujiang Li, Yunan Liu, Yunqing Wang, Yuntao Li, Yurong Wu, Yutao Zhang, Yuxi Duan, Yuxuan Zhang, Zezhen Liu, Zhengtao Jiang, Zhenhe Yan, Zheyu Zhang, Zhixiang Wei, Zhuo Chen, Zhuoer Feng, Zijun Yao, Ziwei Chai, Ziyuan Wang, Zuzhou Zhang, Bin Xu, Minlie Huang, Hongning Wang, Juanzi Li, Yuxiao Dong, and Jie Tang. Glm-5: from vibe coding to agentic engineering, 2026.

[26] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. Bertscore: Evaluating text generation with bert, 2020.

[27] Howard Levene. Robust tests for equality of variances. In Ingram Olkin, Harold Hotelling, et al., editors, *Contributions to Probability and Statistics: Essays in Honor of Harold Hotelling*, pages 278–292. Stanford University Press, Stanford, CA, 1960.

[28] Norwegian University of Life Sciences (NMBU). Guidelines for the use of artificial intelligence (AI) at NMBU, 2024. Accessed: 2026-04-07.